THREE-HOUR TEACHING WORKSHOP · OCTOBER 2026

Agentic AI for Grid Operators

Concepts, executable workflows and evidence
Voltage support · Restoration · Cyber response

> This handout follows every teaching slide in order. Explanations, equations, code excerpts, discussion questions and expected observations are integrated with the three cases. There is no appendix.
>
> Use the animated Slidev deck during the lecture and this document for study. Slide numbers below match the presentation.

What you will be able to do

- Explain dependencies, state, bounded repair and human checkpoints.
- Assign a numerical method and an evidence contract to each agent stage.
- Run and interpret the engineering baselines before judging language-model contributions.
- Read a recommendation with its model scope, constraints, rejected alternatives and latency.

The voltage example uses one synthetic SMART-DS Greensboro feeder. Restoration and cyber classroom exercises use separate small fixtures. The tutorial is advisory and makes no claim of operating utility equipment.

Learning route and preparation

Minutes	Teaching and experiment	Notebook checkpoint
0–40	Agent concepts, research, DAGs, real LangGraph and contracts	G0–G5, L0–L1
40–82	Voltage support, MPC, SAC, Llama, model comparison	L2–L5
82–92	Break	Save work
92–119	Restoration MILP, critical-load priorities, stale evidence	L6–L7
119–149	WLS, stealth attacks, protected-data response	L8–L9
149–158	Research frontier, matched comparisons, latency	L11 optional live call
158–180	Five-stage responses, control-room review, capstone	L10, L12

Prepare environments before class. Use Python 3.12 and start Jupyter from the tutorial folder. The graph notebook and engineering notebook have separate environments so that graph-runtime dependencies do not change the recorded scientific environment.

Engineering environment

```
python -m venv .venv-grid
# Activate the environment for your operating system.
python -m pip install -r requirements_linear.txt
python -m jupyter lab Classroom_Labs.ipynb
```

Graph environment

```
python -m venv .venv-graph
# Activate this separate environment.
python -m pip install -r requirements_graph_lab.txt
python -m jupyter lab Graph_Workflow_Lab.ipynb
```

The engineering default performs fresh numerical calculations and saved SAC actor inference, then reads genuine archived language-model responses. It does not download or retrain a model. For the optional full OpenDSS path, install `requirements.txt` and set `RUN_OPENDSS=True` in L0. Run OpenDSS calls sequentially. For L11, prepare the existing Ollama service with `llama3.1:8b` and set `RUN_LOCAL_LLAMA=True`. Keep model downloads outside class.

The completed graph notebook and engineering notebook are also available as reading pages. The graph notebook executes a real LangGraph runtime on a declared toy model. The engineering workflow uses the existing custom Python supervisor. The graph lesson teaches orchestration and does not migrate the numerical studies.

Reading the evidence

Fresh numerical experiment, archived LLM reply and new live call are different evidence types. Retain the scenario, candidate, model scope and timestamp when comparing them. A hash detects a changed record but does not authenticate a measurement. A solver convergence flag does not establish all operating limits. A concise model explanation is a proposed rationale that still needs numerical checking.

Notebook discipline

Run each notebook in cell order. Change one factor at a time and preserve the original result. For the LangGraph lab, keep G4's three baseline assertions unchanged and put altered scenarios in a new scratch cell. In G5, pause after the interrupt result to inspect it before supplying the mock readback. The completed G5 cell includes that simulated readback automatically. No real operator identity is authenticated.

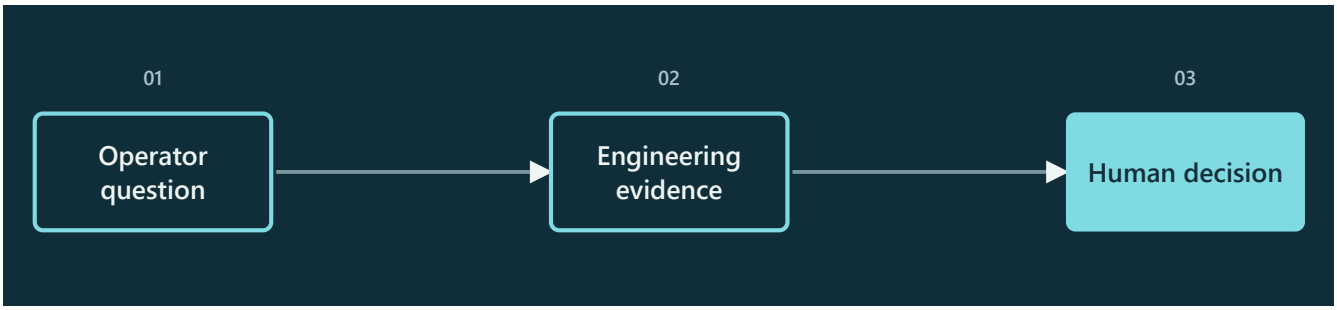
Navigation: Space or Right Arrow advances a click; Left Arrow reverses it. Slidev presenter mode exposes teaching notes. Escape opens the overview. The PDF slides show completed states; animation requires the browser presentation.

Foundations

SLIDE 1 · F01

Agentic AI for grid operators

> **Build an evidence trail before asking an operator to act.**



Completed teaching diagram.

- Understand the workflow
- Run the engineering tools
- Challenge the recommendation

Begin with the operator's task rather than the software package. A recommendation becomes useful when someone can identify its inputs, inspect the calculation, and understand why it passed or failed a declared check. Ask the audience to keep one recurring question in mind: what evidence would change your decision? The workshop connects graph execution, engineering models, learned controllers, language-model responses, and human review. Each component has a different responsibility. We will use an illustrative graph first, then the existing numerical notebooks. The feeder study is a synthetic Greensboro example, not a live utility deployment. Reveal the three learning actions individually. Transition: an operator encounters three kinds of decisions, and the same evidence discipline must survive all three.

PowerAgent roadmap · IEEE PEM 2025

Next: Three decisions; one discipline

SLIDE 2 · F02

Three decisions; one discipline

> **The mathematical task changes; the requirement for traceable evidence remains.**

Operations discussed: Voltage support, Restoration, Cyber response.

- Voltage: choose a continuous inverter command
- Restoration: choose permitted paths and served loads
- Cyber response: decide which measurements support control

Invite one short answer before revealing the three cases. Voltage support concerns continuous active and reactive control. Restoration includes discrete decisions and load priorities. Cyber response concerns the reliability of the information used to estimate and control a state. These are not three names for the same optimization problem. The tutorial therefore changes its numerical method while retaining a shared record and explicit verification. Keep the question grounded: an operator needs evidence for this candidate in this case, not a general claim that the language model is capable. Later results will include ties, rejected commands, and a model-scope mismatch. Transition: turn these practical needs into three things participants should be able to do at the end.

> **Discussion**
>
> What must an operator see before accepting any of these?

Expected observation: The exact candidate, declared model, checked limits, evidence age, and unresolved assumptions.

Tutorial notebooks · three declared case scopes

Next: What you will be able to explain

SLIDE 3 · F03

What you will be able to explain

> **Explain the graph, reproduce a decision, and defend an acceptance or hold.**

- Trace state through every proposed stage
- Match code, equations, outputs, and timing boundaries
- Separate numerical benefit from workflow benefit

State the assessment criteria plainly. A successful participant can point to the node that produced a value, connect a displayed equation to the notebook operation, and explain the final review status. Do not equate the presence of several agents with improved control. The workflow might organize evidence better while producing the same command as a conventional program. Conversely, it might spend more time without improving either result. This is why we separate numerical quality, trace quality, and latency. The labs deliberately include an ordinary-Python baseline and matched engineering comparisons. Use the audience question to make a null numerical result scientifically useful. Transition: show when participants will learn each concept and when they will run it.

> **Discussion**
>
> If the final objective ties the baseline, what could still improve?

Expected observation: Inspectability or stale-evidence rejection may improve; those benefits need their own measurements.

PowerAgentBench-SS · 2026 preprint

Next: A three-hour route with experiments throughout

SLIDE 4 · F04

A three-hour route with experiments throughout

> **Each case follows concepts, an executable experiment, and an evidence-based discussion.**

Block	Work at the keyboard
Foundations	G0–G5; L0–L1
Voltage	L2–L5; archived and optional live Llama
Restoration / cyber	L6–L9
Evidence / operator	L10–L12

- 0–40 min: foundations and graph lab
- 40–82 min: voltage; 82–92 min: break
- 92–149 min: restoration and cyber response
- 149–180 min: evaluation and operator capstone

The course is a single teaching sequence; the technical explanations and exercises are part of the lecture. Software installation is a prerequisite, not an activity to begin during the first exercise. Ask participants to open both notebooks now while keeping their current kernels separate. The graph notebook teaches orchestration with transparent arithmetic. The classroom notebook performs the engineering experiments and reads explicitly labeled archived language-model responses. A fresh calculation, a saved actual response, and a newly generated model response are different evidence modes. Preserve those labels when discussing speed or quality. The schedule reserves time for prediction and explanation rather than continuous lecturing. Transition: before opening the first code cell, identify what makes a workflow agentic.

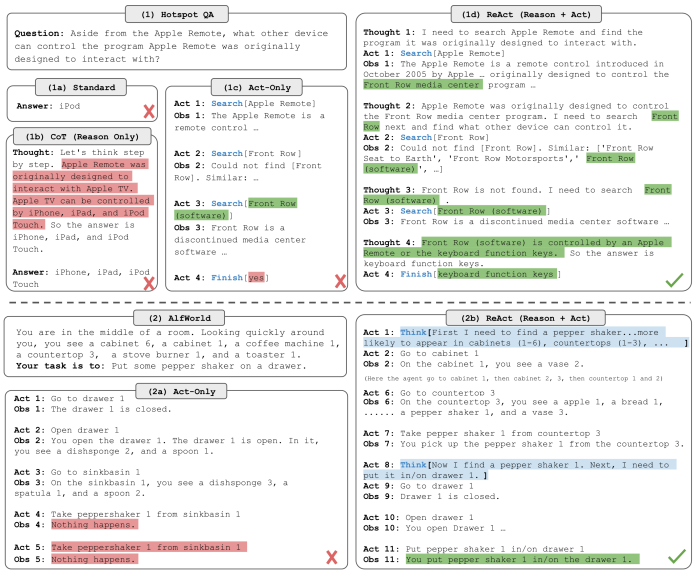
Graph laboratory · Engineering laboratory

Next: A useful agent closes a feedback loop

SLIDE 5 · F05

A useful agent closes a feedback loop

> **A returned tool result can change the next permitted operation.**



Yao et al. (ICLR 2023), ReAct Fig. 1, CC BY 4.0. Unchanged figure.

- Observe the current evidence
- Select an allowed operation
- Read its result before continuing

Reveal the loop one action at a time. ReAct is an influential example of combining language-model deliberation with external actions and observations. Our teaching emphasis is the observable interface: what operation was requested, what result came back, and how the next decision used it. We do not need access to private model reasoning to inspect those steps. A predefined workflow follows known routes; an agent can choose among permitted operations using the current context. Real applications can mix both patterns. In this tutorial, deterministic routing and numerical limits constrain language-selected trials. A loop does not by itself establish reliability or intelligence. Transition: separate the component choosing an operation from the component calculating its result and the runtime coordinating both.

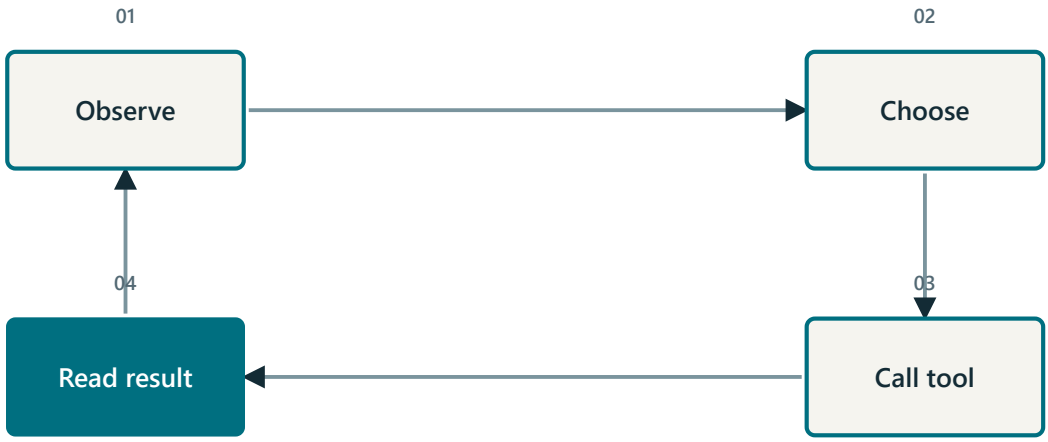
Yao et al. · ReAct · ICLR 2023 · LangGraph · workflows and agents · Figure and reuse license

Next: Tool feedback changes the next choice

SLIDE 6 · F05A

Tool feedback changes the next choice

> **The useful loop is a decision updated by an observation.**



Completed teaching diagram.

- A choice requests a bounded tool.
- The tool returns external evidence.
- New evidence can change the next choice.

This original teaching diagram adapts the interaction pattern explained by ReAct. It replaces the published question-answering example with four neutral operations. Start at the current observation, identify an allowed tool action, execute that action, and read the returned evidence before choosing again. The arrows show information flow. They do not guarantee that the next choice is correct. In the grid tutorial, a numerical verifier may reject the proposed action even when the language explanation sounds plausible. The animation highlights one operation at a time on the same slide. A deterministic retry budget and explicit exit conditions are introduced later, because an unbounded feedback loop can waste computation. Transition: separate the agent making a choice from the tool doing the calculation and the orchestrator enforcing the route.

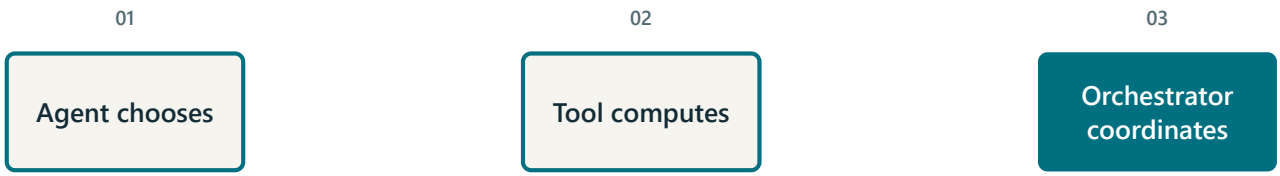
Conceptual adaptation: Yao et al., ReAct (ICLR 2023), CC BY 4.0

Next: Tool, agent, orchestrator: three different jobs

SLIDE 7 · F06

Tool, agent, orchestrator: three different jobs

> **Roles describe responsibilities; they do not require different language-model backbones.**



Completed teaching diagram.

- Agent: chooses an allowed next operation
- Tool: returns a specific calculation or record
- Orchestrator: runs routes and stores state

Use a concrete voltage example. A planning role proposes a candidate, a power-flow tool evaluates it, and the runtime requires a verification step before the candidate is eligible for review. The same Llama model can support several role-specific prompts. Numerical functions can perform a role without calling any language model. This distinction prevents a common confusion: five boxes are not necessarily five independent models, five independent opinions, or five levels of protection. The responsibilities are useful because their input and output contracts can be inspected separately. The engineering tutorial implements its orchestration in custom Python; the graph lab introduces LangGraph mechanics separately. Transition: a tool becomes dependable only when its request and response are sufficiently explicit to check.

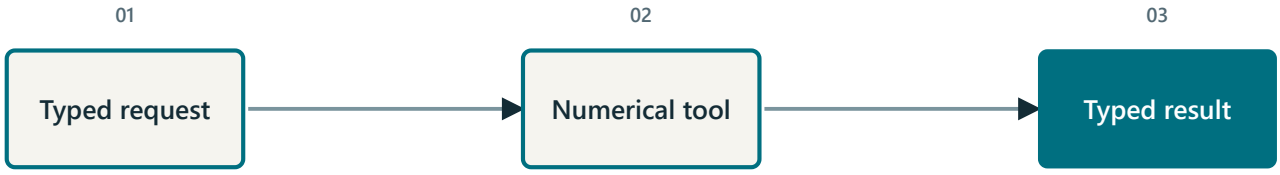
Wu et al. · AutoGen · 2023 preprint · Tutorial implementation

Next: A tool call is a small engineering contract

SLIDE 8 · F07

A tool call is a small engineering contract

> **A result needs a candidate, units, model scope, and status.**



Completed teaching diagram.

- Input: case, revision, command, and limits
- Output: values, units, convergence, and evidence identity
- Failure: explicit error or rejected result

Treat the tool contract like a laboratory measurement sheet. The number 112 is unusable until its unit, location, candidate, and calculation conditions are known. A solver may converge to a state that violates an operating limit. A request may be well formed yet refer to an outdated network. The contract makes these distinctions available to later nodes; it does not resolve them automatically. The tutorial records the candidate and declared context so that verification can inspect a particular calculation. Ask participants to identify which fields a current-limit check needs. Reveal the answer only after someone distinguishes convergence from feasibility. Transition: standardized tool communication can make these contracts reusable across software, but the engineering checks still belong to the application.

> **Discussion**
> Does “solver converged” mean the current limit passed?

Expected observation: No. Convergence and operating feasibility are separate checks.

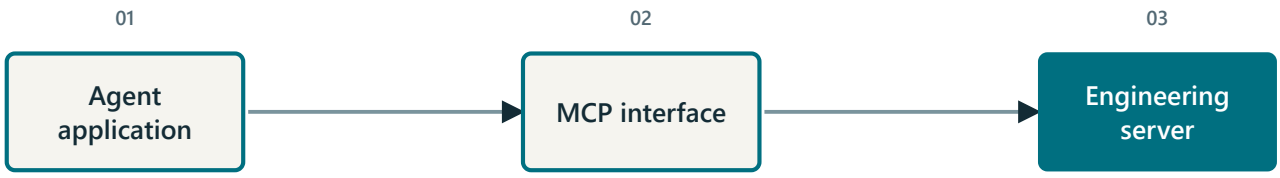
MCP tools specification · 2025-11-25 · Classroom evidence functions

Next: Where MCP fits in the picture

SLIDE 9 · F08

Where MCP fits in the picture

> **A common tool interface helps integration; it does not certify the returned engineering result.**



Completed teaching diagram.

- Discover available tools
- Submit structured arguments
- Receive a structured result

Explain Model Context Protocol as an interface layer that exposes tools to an application. A consistent call format can reduce integration work when different engineering packages are involved. The diagram is architectural context, not a claim that our local classroom notebook uses an MCP server. Its functions are called directly in Python. This is a useful distinction between understanding an ecosystem and reproducing a specific implementation. Ask which part would still need to know whether 112 amperes exceeds the permitted rating. That comparison remains application logic using an agreed unit and limit. A transport protocol cannot supply a missing model assumption. Transition: now place the proposal's five responsibilities above the particular tool and transport choices.

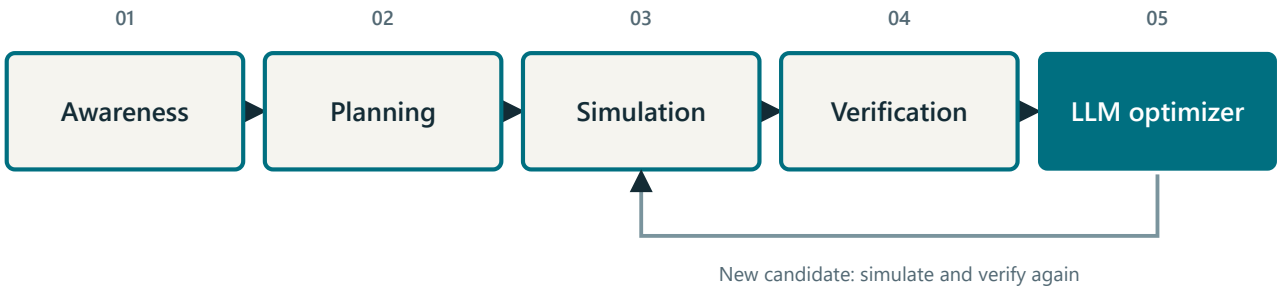
MCP tools specification · 2025-11-25 · PowerMCP · official repository

Next: The complete proposed workflow has five stages

SLIDE 10 · F09

The complete proposed workflow has five stages

> **Every changed candidate must return to simulation and verification.**



Completed teaching diagram.

- Awareness establishes state and request
- Planning proposes; simulation computes consequences
- Verification checks; optimization chooses the next trial
- Operator review receives supported evidence

Use the animation to carry one candidate through all five proposed stages. Awareness establishes the case and interprets the request. Planning chooses a numerical formulation and produces a candidate. Simulation calculates modeled consequences. Verification checks the declared constraints and evidence consistency. The LLM optimizer proposes or selects permitted follow-up work, which must be evaluated again if it changes the action. Human review is the handoff after these responsibilities, not a replacement for a missing stage. All five are represented in the archived engineering trace later in the course. Some work is deterministic, some uses numerical optimization, and some uses model responses. Transition: identify which responsibility the language model can help with, and which remains a numerical calculation.

Executed five-stage tutorial implementation · Five-stage archived evidence

Next: Give each stage the method it needs

SLIDE 11 · F10

Give each stage the method it needs

> **The language model coordinates bounded choices; numerical methods produce and check the physical evidence.**

Responsibility	Returned artifact
Llama interpretation	Allowed operation + public explanation
Numerical method	Candidate + calculated values
Verifier	Pass/hold/reject + margins

- Planning: MPC, restoration MILP, or estimation/control
- Simulation: LinDistFlow, OpenDSS, or declared teaching model
- Verification: explicit numerical and consistency checks
- Selection: fixed rule, vote, or learned bandit

Avoid presenting the same algorithm as the natural solution to every case. Continuous voltage support suits the MPC and SAC comparison. Restoration uses a mixed-integer service-allocation formulation. The cyber teaching example combines estimation, anomaly evidence, and a constrained response. These methods are part of the workflow rather than competitors excluded by its design. The language model sees a bounded catalog of permitted operations and returns a choice with a public explanation. The explanation is checked against the numerical evidence; it is not treated as a proof. Learned response selection is a separate, discrete decision from SAC's continuous physical control. Transition: before reading the code, establish a visual vocabulary that clearly separates computation from the electrical network.

Classroom methods · Proposal case tools

Next: Learn four symbols before reading the graph

SLIDE 12 · F11

Learn four symbols before reading the graph

> **Workflow arrows represent dependencies, not electrical connections.**

A	An operation or node	s	The current record
$A \rightarrow B$	B requires the result from A	$r(s)$	A routing rule evaluated on state

- Box A: an operation
- Record s: current shared state
- Arrow A → B: B needs A's result
- Diamond r(s): choose the permitted next route

Point to each visual symbol as it appears. A node is a unit of work. State is the record that subsequent work reads. A directed edge identifies an allowed execution dependency. A routing rule chooses a successor based on the current record. Use the same symbol meanings when the diagram becomes code later. The letters here are computational names; they do not identify buses, currents, or physical switches. This matters because later restoration figures also contain nodes and edges, but those edges represent network connections. Ask the audience what an arrow from simulation to verification requires: the verification operation must have a result to inspect. Transition: begin with the simplest dependency structure, one with no path returning to its starting node.

LangGraph graph API

Next: DAG means directed and acyclic

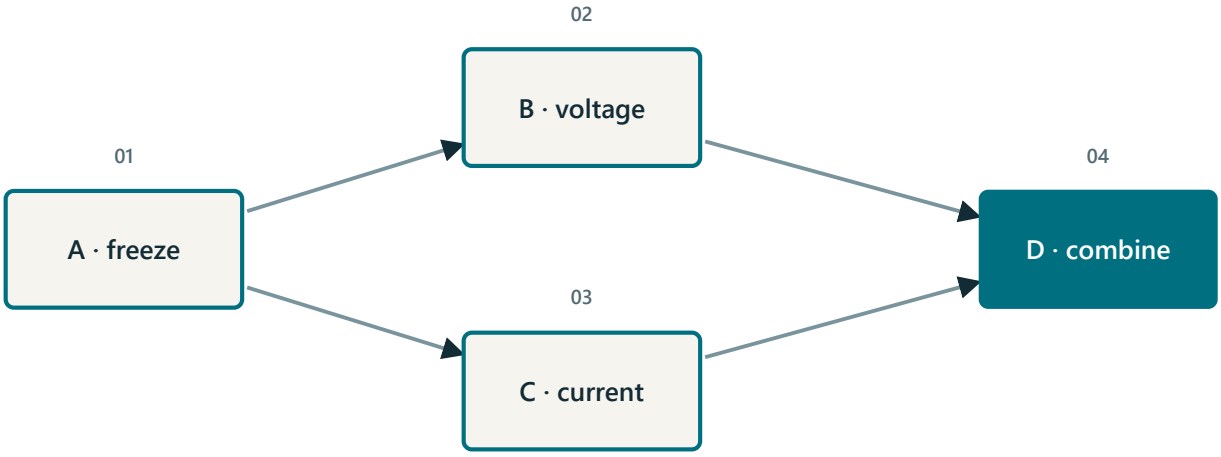
SLIDE 13 · F12

DAG means directed and acyclic

> **A DAG encodes dependencies without a backward path.**

$$G = (N, E), \qquad N = \{A, B, C, D\}$$

G	Dependency graph	N	Set of operations
E	Set of directed prerequisite edges		



Dependencies determine which tools may run together

Completed teaching diagram.

- A freezes the input
- B and C inspect independent results
- D combines both pieces of evidence

Build the diamond-shaped dependency diagram in place. The example freezes a candidate before checking its voltage and current evidence. The final operation needs both check results. Directed means arrows have an orientation; acyclic means no sequence of arrows returns to its starting operation. Define the node set and edge set without requiring graph theory background. The two middle checks are conceptual operations on already produced evidence. This picture does not claim that two OpenDSS engines run concurrently, and it does not describe the cyclic graph used in the G lab. Let the audience trace each prerequisite with a finger. Transition: more than one valid execution order can preserve the same arrows, which gives us a short prediction exercise.

NetworkX · topological sorting

Next: Which schedule respects every arrow?

SLIDE 14 · F13

Which schedule respects every arrow?

> **The join must follow both prerequisites, whatever their order.**

Operations discussed: A, B, C, D.

Order	Valid?
A → B → C → D	✓
A → C → B → D	✓
A → B → D → C	✗

- A, B, C, D
- A, C, B, D
- A, B, D, C

Pause before revealing the validity marks. Ask participants to justify an answer by naming an edge, not by guessing the intended order. Both B and C depend on A, and D depends on both B and C. Therefore the middle operations may exchange positions, while the final operation cannot move ahead of either prerequisite. This is the meaning of a topological order. It describes a valid ordering, not a measured time or speedup. A real scheduler also has resource limits, tool state, and failure handling. If a participant proposes concurrent middle operations, acknowledge that dependencies permit it only when the implementation and resources permit it too. Transition: inspect what must be preserved when work actually branches and later rejoins.

> **Discussion**
>
> Which two orders are valid?

Expected observation: The first two. The third starts D before C has supplied its result.

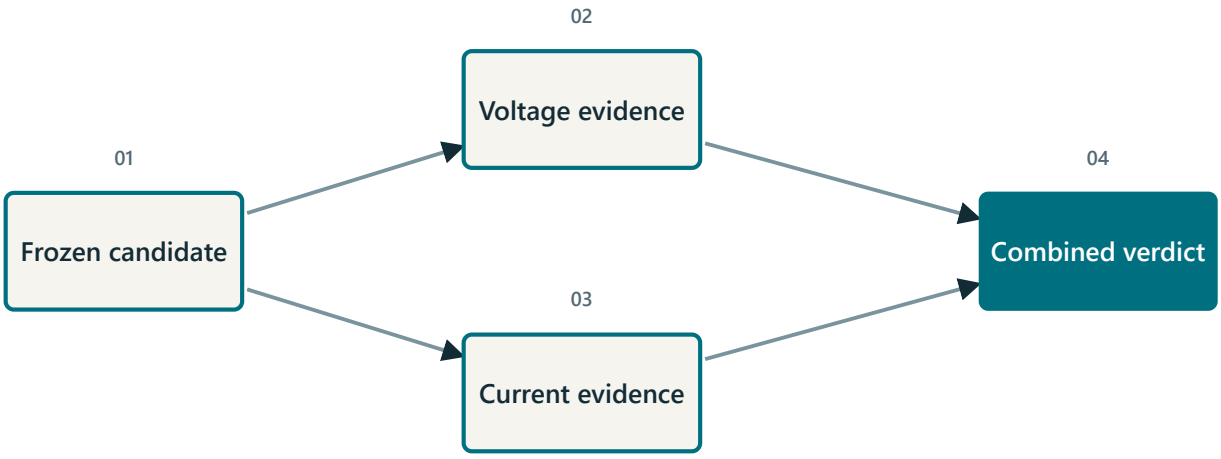
NetworkX · topological sorting

Next: Fan-out and join need a shared identity

SLIDE 15 · F14

Fan-out and join need a shared identity

> **Both branches must refer to the same frozen candidate and context.**



Dependencies determine which tools may run together

Completed teaching diagram.

- Fan-out copies a consistent input
- Each branch returns its own evidence
- Join checks completeness and identity

Animate the same candidate identifier into both branches. Then change the identifier on one returned record conceptually and ask whether the results may still be combined. They may not establish a verdict for one common candidate. This is a simple way to teach stale or mismatched evidence without introducing complex concurrency machinery. Parallel execution can reduce elapsed time when work and resources are independent, but it can also create synchronization and merge problems. The tutorial's engineering runs should not be described as parallel simply because this diagram branches. Their timing boundary is measured separately. The visual teaches a design obligation: completeness and identity at the join. Transition: a failed verification adds another kind of edge, one that deliberately goes back for a revised calculation.

LangGraph graph construction examples · Tutorial evidence binding

Next: A repair loop is a cycle, with a budget

SLIDE 16 · F15

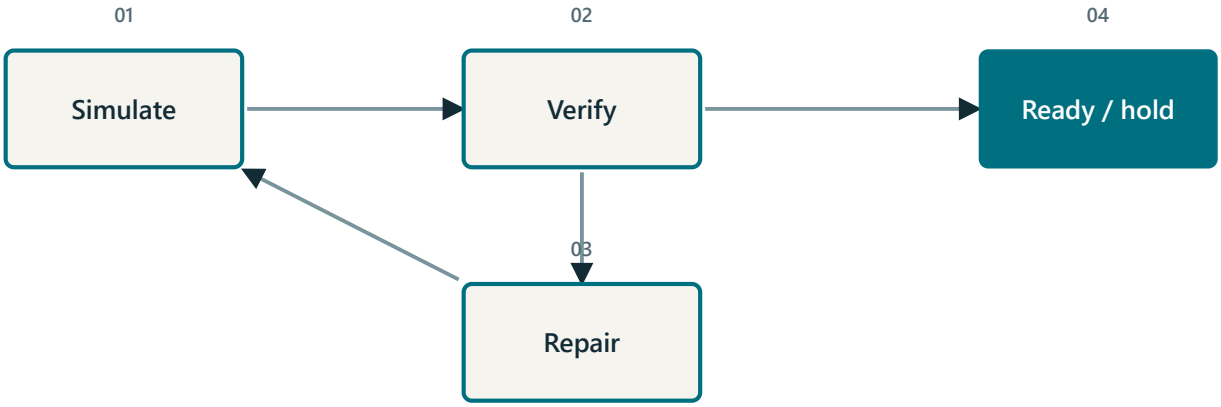
A repair loop is a cycle, with a budget

> **A finite retry budget bounds attempts; it does not guarantee a feasible command.**

$$\text{repair only if } k < K$$

k Completed repairs

K Maximum permitted repairs



Bounded repair budget • exhausted budget routes to HOLD

Completed teaching diagram.

- Pass → ready for review
- Fail with budget → repair and simulate again
- Fail without budget → hold

Trace the success path first, then reveal the backward repair edge. Once that edge exists, the program graph is cyclic and should not be called a DAG. A finite individual execution can still be drawn as a sequence of visits. The two integer symbols count used and allowed repairs. The tutorial increments the counter when it modifies the command and sends that command back through simulation. A finite count guarantees that this policy eventually stops if its tool calls return; separate timeouts would be needed for a stalled tool. Holding is a valid outcome when no admissible command is found within the budget. Transition: the router needs both the latest verdict and the counter, so these values must live in shared state.

Executed retry policy · G2–G4

Next: State is the laboratory record carried forward

SLIDE 17 · F16

State is the laboratory record carried forward

> **A node reads state and returns explicit updates to the record.**

$$s_{t+1} = \mathrm{merge}(s_t, \Delta s_t)$$

s_t	Record before workflow step t	Δs_t	Returned field updates; not arithmetic subtraction

- Candidate: command and context
- Evidence: current, limit, and verdict
- Control flow: attempts, budget, and status
- Trace: the sequence of recorded events

Read the displayed record as a laboratory notebook with named fields. The command identifies what was proposed, the evidence fields identify what was calculated, and the control-flow fields identify what should happen next. The merge expression is a conceptual description of applying returned updates. Delta is not a numerical difference between two entire records. The G notebook uses TypedDict to describe expected keys and types, while its initial-state function explicitly validates numerical inputs. TypedDict itself does not perform runtime value checking. A chat transcript alone is not a substitute for this engineering record because important values and units must be easy to locate and compare. Transition: different fields need different rules when new information arrives.

G1 state schema · Python TypedDict

Next: Replace a result; append an event

Replace a result; append an event

> **The merge rule is part of the data model.**

$$R_{\text{trace}}(a, b) = a + b$$

R	Reducer combining stored value and update	a, b	Existing event list and new event list

- Current: 1.00 then 0.80 → keep 0.80
- Trace: [simulate] plus [verify] → keep both
- Parallel writes need an explicit compatible rule

Use the animation to replace the scalar value while visibly extending the event list. The current field describes the latest simulated command, so retaining the old value there would be misleading. The trace field describes history, so discarding earlier events would make the repair difficult to inspect. The example's plus symbol means list concatenation, not arithmetic averaging or consensus. In the code, Annotated attaches the addition function to the trace field. An empty appended list does not erase existing events. If several branches write a field, the application needs a merge rule appropriate to that field and its ordering semantics. Transition: after updating the record, one small routing function can choose among success, repair, and hold.

> **Discussion**
>
> Should a repaired command keep the previous current as its current result?

Expected observation: No. Replace the current result; preserve the previous event in the trace.

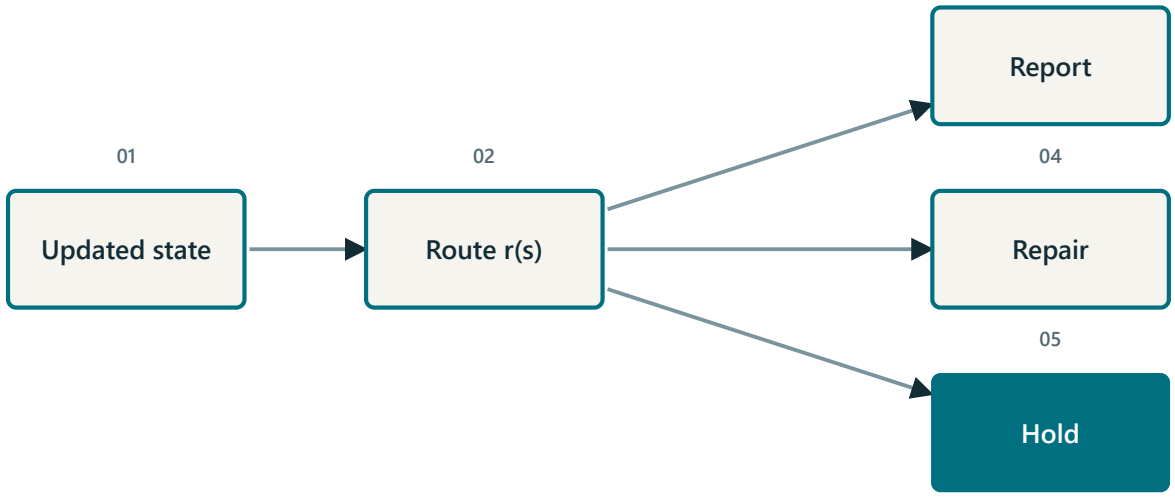
G1 · list-addition reducer · LangGraph · reducers

Next: Routing turns evidence into the next step

SLIDE 19 · F18

Routing turns evidence into the next step

> **Verification computes the verdict; routing chooses what to do with it.**



Completed teaching diagram.

- passed = true → report
- passed = false and k < K → repair
- passed = false and k ≥ K → hold

Reveal the mutually exclusive conditions in order. The first condition checks whether the latest calculation passed. If it did not, the second decision is whether a repair is still permitted. The route returns a label, and the graph maps that label to a node. This policy is deliberately deterministic so that participants can reason about every execution path. A more flexible language-selected operation could be inserted elsewhere without allowing it to bypass these acceptance conditions. Ask what would happen if the counter were equal to the budget: the route must hold. The numerical verification rule and the orchestration policy remain separate functions. Transition: a report can itself pause for review, which requires saved execution state.

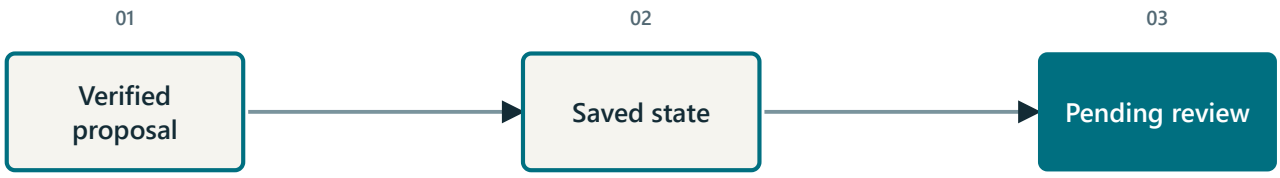
Exact route function · G2

Next: Checkpoint: remember where the workflow stopped

SLIDE 20 · F19

Checkpoint: remember where the workflow stopped

> **A checkpoint associates execution state with a thread identifier.**



Completed teaching diagram.

- Save the current record
- Keep the next node
- Resume the same execution context

Use a bookmark analogy, then make it precise with the stored record and pending node. In the G lab, a thread identifier associates the checkpoint with a particular execution. That identifier is not an operator credential. The example uses InMemorySaver, so the state remains available in the running process but does not provide durable recovery after the kernel disappears. This scope is sufficient for observing a pause and resume during class. Participants should inspect the snapshot before discussing its final status; otherwise a whole-cell execution can make the pause easy to miss. The upcoming exercise prints the pending node and acknowledgment flag. Transition: an interrupt exposes the question that a human or caller must answer before that execution can continue.

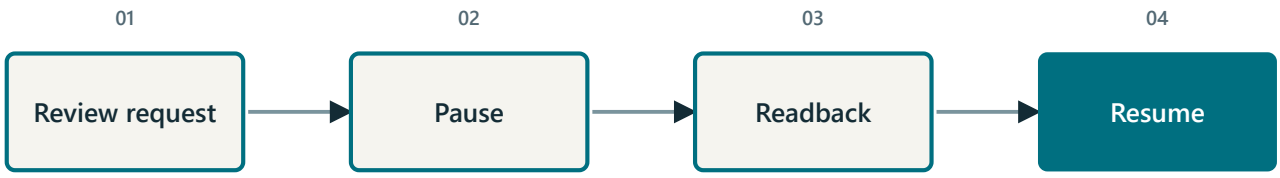
LangGraph · persistence · G5 · InMemorySaver example

Next: Interrupt: make the review decision visible

SLIDE 21 · F20

Interrupt: make the review decision visible

> **A review response must identify the recommendation it acknowledges.**



Completed teaching diagram.

- Pause with candidate and evidence scope
- Inspect before responding
- Resume with an explicit matching response

Distinguish a visible recommendation from an acknowledged recommendation. The toy sends an interrupt payload that includes a recommendation identifier, command, calculated value, and declared scope. Its scripted reply must approve the same identifier. A wrong identifier or a declined response remains distinguishable from approval. This demonstrates a record-consistency mechanism, not operator authentication or equipment dispatch. On resumption, the interrupted node can begin again, so uncontrolled side effects before the interrupt would be a design problem. The toy only records state changes. Ask participants what the operator should see if the candidate changes while review is pending. The answer motivates fresh evidence and a new identity. Transition: organize these requirements into three checks that answer different questions.

LangGraph · interrupts · G5 · simulated readback

Next: Typed, current, feasible: three separate claims

SLIDE 22 · F21

Typed, current, feasible: three separate claims

> **Passing one check does not establish the others.**

Check	A pass does not prove
Schema ✓	Correct intent interpretation
Binding ✓	Source authenticity
Engineering ✓	Adequate model or field authorization

- Schema: are required values valid?
- Binding: is this result for this candidate?
- Engineering: do modeled limits pass?

Keep this slide concrete rather than reading a compliance checklist. Give the current example verbally and ask for the verdict before revealing it. A converged solver result can fail a physical operating condition. A correctly shaped result can be stale. A matching hash can identify consistent data without proving who produced it. These checks are complementary because they answer different questions. Our later lab uses an explicitly illustrative payload to isolate the binding mechanism; participants must not mistake its passed field for a real feeder calculation. The final operator packet adds another distinction between evidence scope and the requested scope. Transition: ask what happens when a favorable language-model explanation conflicts with a failed numerical check.

> **Discussion**
>
> The solve converged at 112 A; the limit is 100 A. Verdict?

Expected observation: Reject that candidate under the declared current limit.

Tutorial binding and operator checks

Next: A vote cannot raise a declared limit

SLIDE 23 · F22

A vote cannot raise a declared limit

> **Selection chooses a trial; verification determines whether it is admissible.**

Operations discussed: Model responses, Selected trial, Numerical check, Reject / retain.

- Two models favor a candidate
- The numerical check reports 112 A
- The declared limit remains 100 A

Use the fictional numbers as a fast diagnostic question. The source of a candidate and the number of models supporting it do not change its calculated current or the stated limit. The selected operation has earned a trial, not automatic acceptance. A failed trial should leave the previous admissible candidate available and should produce an inspectable rejection record. The later voltage experiment implements this separation between response selection and numerical evaluation. Multiple models may also share errors, so agreement does not establish independent evidence. This lesson is compatible with both conventional supervisors and agentic workflows. Transition: close the concept sequence by showing which existing research frameworks contribute related mechanisms, using qualified claims rather than a promotional ranking.

> **Discussion**
>
> Can a majority vote make this candidate acceptable?

Expected observation: No. Reject it, preserve the incumbent, and record the failed margin.

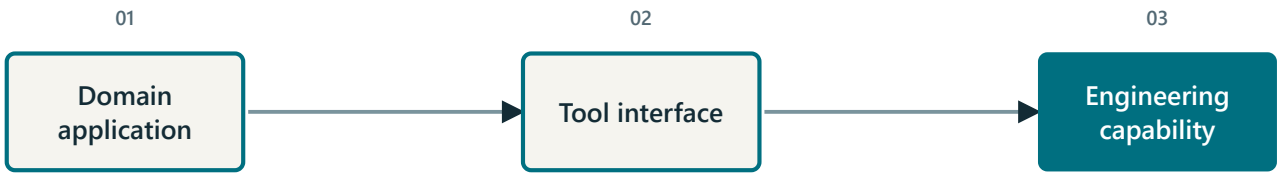
Tutorial candidate verification · Grid-Agent · sandbox validation

Next: PowerAgent: connect engineering capabilities

SLIDE 24 · F23

PowerAgent: connect engineering capabilities

> **Reusable tool interfaces and workflows reduce integration effort across applications.**



Completed teaching diagram.

- PowerMCP exposes engineering tools
- PowerWF organizes reusable workflows
- Capabilities vary with the selected application

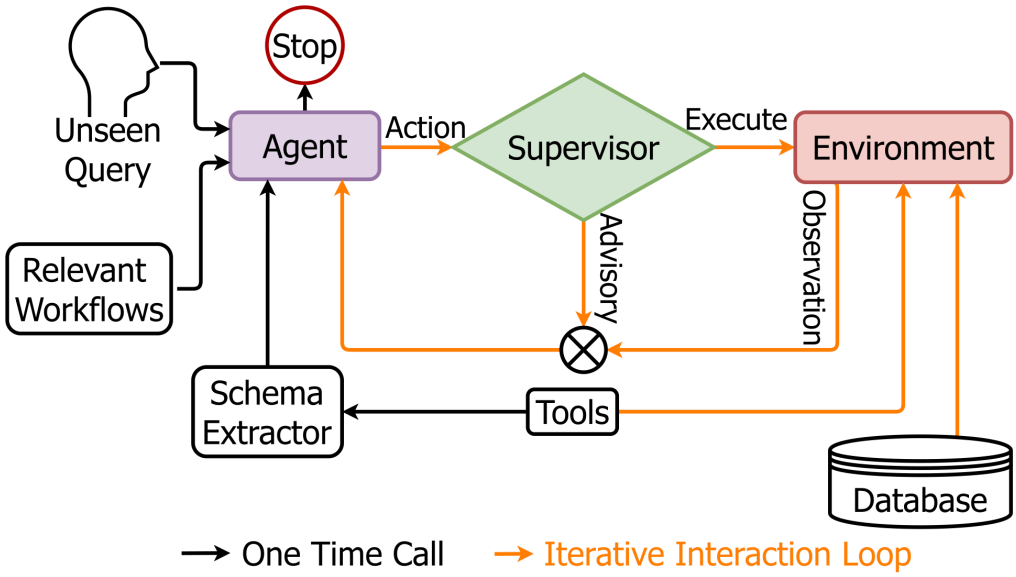
Present PowerAgent as a roadmap and ecosystem rather than one immutable controller. Its official materials organize domain capabilities through tools and reusable workflows. The teaching connection is our tool-contract slide: a clearly defined interface can make the same numerical capability usable from different applications. The diagram on this slide is a tutorial schematic of that concept, not a reproduced architecture figure. Do not infer an ecosystem-wide acceptance guarantee from the availability of a tool. In class, ask which fields an OpenDSS wrapper should return so that a different planner can still use its result. This turns a literature example into a design exercise. Transition: a connected collection of tools still needs protection against using them in an invalid order.

Zhang & Xie · PowerAgent · IEEE PEM 2025 · PowerAgent · official ecosystem · PowerMCP

Next: PowerDAG: check prerequisite relationships

PowerDAG: check prerequisite relationships

> **A one-shot advisory detects a covered prerequisite violation; repeated calls may proceed.**



Badmus & Pandey (2026), Fig. 2, CC BY-SA 4.0. Unchanged figure.

- Retrieve relevant workflow exemplars
- Check prerequisite tool writes within covered scope
- One-shot advisory; repeated calls may proceed

PowerDAG combines exemplar retrieval with deterministic checks of selected tool prerequisites. Its inspected study reports 30 unseen queries, 50 expert exemplars, and 82 tools. Section IV-C2 describes a one-shot advisory: an initial covered violation is blocked, but repeating the call is permitted. This is not an unconditional hard safety gate. Relate the mechanism to our fan-out identity example while retaining that distinction. The cited figure exposes the paper architecture; our classroom code illustrates a narrower evidence-binding idea. Ask what our own application should require after the load changes: evidence associated with the new context. Our guard rejects stale binding explicitly. Transition: the next framework emphasizes what happens after a proposed physical change fails in a sandbox.

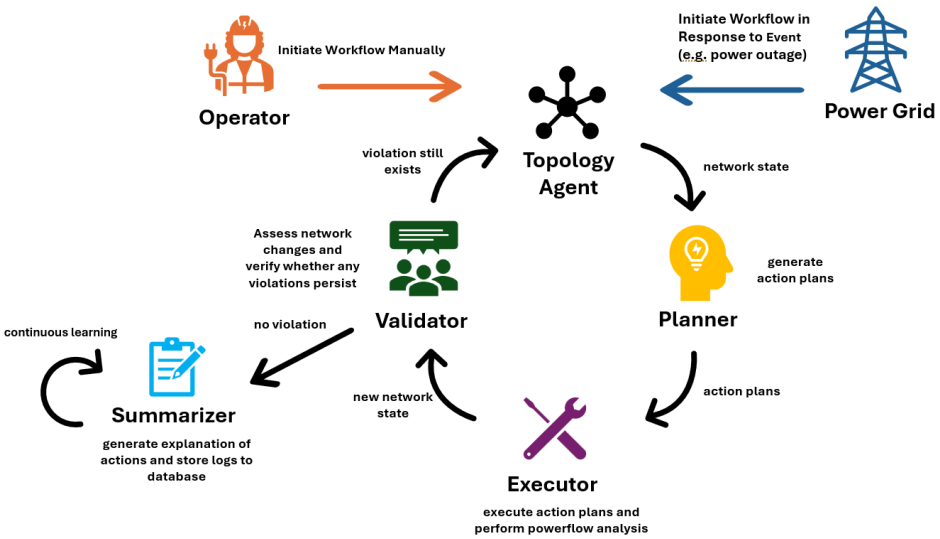
Badmus & Pandey · PowerDAG · 2026 preprint · Figure and reuse license

Next: Grid-Agent: test a change before retaining it

SLIDE 26 · F25

Grid-Agent: test a change before retaining it

> A failed trial should not silently replace the incumbent.



Zhang et al. (2025), Grid-Agent v3, Fig. 1, CC BY 4.0. Unchanged.

- Topology, planner, executor, validator, summarizer
- Evaluate actions in a simulation sandbox
- Retain or roll back using returned evidence

The inspected Grid-Agent version describes five roles and a sandbox workflow with validation and rollback. Its role names are Topology, Planner, Executor, Validator, and Summarizer. This is close prior art for splitting grid work among specialized responsibilities, so our teaching workflow should not claim to invent five-agent grid verification. The useful classroom question is whether a rejected candidate can accidentally become the retained command. Use the paper figure to locate the validation loop, then connect it to our separate incumbent and trial records. The v3 source recommends human oversight for deployment; it does not establish that every demonstrated action underwent authenticated operator approval. Transition: another architecture separates planning from coordination and domain-tool execution, which helps explain why state handoffs matter.

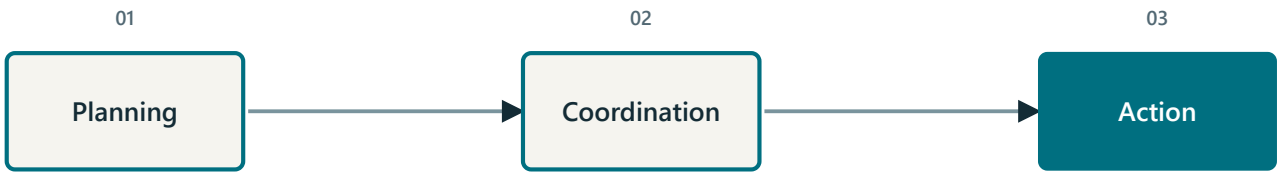
Zhang et al. · Grid-Agent · v3, 2025 preprint · Figure and reuse license

Next: X-GridAgent: separate planning, coordination, and action

SLIDE 27 · F26

X-GridAgent: separate planning, coordination, and action

> **A hierarchy helps organize work only when each handoff preserves the required context.**



Completed teaching diagram.

- Planning decomposes the requested analysis
- Coordination routes and tracks tasks
- Action selects domain tools and reads results

X-GridAgent organizes power-grid analysis in three layers and exposes domain capabilities through multiple tool servers. Its study uses 11 representative queries repeated 30 times, not 330 distinct tasks. The source also describes retrieval and prompt refinement. For our teaching purpose, focus on one mechanism: a plan needs coordination, and coordination needs an action interface that returns usable evidence. Ask which information must survive all three layers. The candidate, model scope, limits, and task identity are stronger answers than an unstructured summary alone. This slide is an independently drawn teaching schematic of the conceptual hierarchy; it is not an altered paper figure. Transition: compare these mechanisms compactly while preserving the qualifications hidden by a simple checkmark.

Wen & Chen · X-GridAgent · 2025 preprint

Next: Read the ticks as scope, not a league table

SLIDE 28 · F27

Read the ticks as scope, not a league table

> **Cross-paper capability marks do not establish a common performance ranking.**

Work	Tools	Feedback	Checks	Review
PowerAgent ecosystem	✓	✓	△	✓
PowerDAG	✓	✓	△	NR
Grid-Agent v3	✓	✓	✓	△
X-GridAgent	✓	✓	△	△
PowerAgentBench-SS	✓	✓	✓	△
Tutorial	✓	✓	✓	✓

- Different tasks, models, budgets, and evaluation boundaries
- Benchmark work evaluates agents; it is not a controller
- Our benefit must be tested against matched baselines

Define the marks before interpreting the rows. A tick records a documented capability in the inspected source; a triangle preserves a qualification; NR means not reported in that source rather than proved absent. PowerAgent capabilities vary by workflow. PowerDAG covers selected dependencies with a one-shot advisory. Grid-Agent oversight is proposed for deployment. X-GridAgent includes solver checks and human prompt refinement. PowerAgentBench-SS supplies an evaluation protocol with hidden revalidation under a DC scope. The tutorial's review tick refers to local advisory readback, not authenticated field authorization. None of these rows forms a shared performance benchmark. Ask what would make a valid benefit claim: common scenarios, tools, budgets, outcome definitions, and timing boundaries. Transition: run a transparent graph and its ordinary-Python baseline before introducing electrical complexity.

Qualified matrix and source-by-source evidence · PowerAgentBench-SS · 2026 preprint

Next: Our first experiment makes every transition visible

SLIDE 29 · F28

Our first experiment makes every transition visible

> **Learn real graph execution with deliberately simple arithmetic.**

$$i = 0.4 + 0.8u, \qquad i \le L$$

u	Dimensionless command	i	Normalized illustrative current
L	Declared toy limit		

Operations discussed: Simulate, Verify, Repair, Report / hold.

- Simulate the command
- Verify the limit
- Repair by 0.25 or hold

Announce the switch from concepts to the first executable exercise. The response law is invented for teaching; it is not a feeder model, a power-flow approximation, an MPC controller, or a learned policy. Its simplicity lets participants predict each value and inspect every route. LangGraph execution is real, including the conditional edge, checkpoint, interrupt, and resume. The repair subtracts 0.25 from the command without allowing it below zero. The verification compares the resulting illustrative current with the declared limit. We will compare graph execution with a direct-Python loop using identical functions, so the experiment isolates orchestration. Transition: open the notebook and first establish which runtime is actually being used.

G0–G5 · executed graph lab

Next: G0: confirm the runtime and the experiment boundary

G0: confirm the runtime and the experiment boundary

> **Record the installed versions before interpreting the execution.**

Exact excerpt: Graph_Workflow_Lab.ipynb, G0, lines 5–13. Run the complete cell in context.

```
os.environ["LANGSMITH_TRACING"] = "false"
os.environ["LANGCHAIN_TRACING_V2"] = "false"
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command
from src.graph_workflow_lab import initial_state, recommendation_id, summary
versions = {name: version(name) for name in ("langgraph", "langgraph-checkpoint", "langchain-core")}
print(versions)
print("Rule-driven toy graph. No LLM, network call or electrical-model validation.")
```

- Imports the real LangGraph runtime
- Disables external tracing in this lab
- Prints versions and the toy scope

Run the complete G0 cell, even though the slide displays only its relevant lines. Confirm that the version dictionary appears before proceeding. The captured run used LangGraph 1.2.15, checkpoint 4.3.0, and langchain-core 1.6.12; a participant's environment may differ. The cell disables the external tracing flags used by this exercise and declares that no language-model or electrical-model validation occurs. After dependencies are installed, the toy needs no Ollama service. If imports fail, use the supplied graph-lab requirements and return to the documented setup rather than changing the experiment during class. This practical boundary keeps the graph lesson reproducible. Transition: define the record that every subsequent node will read and update.

G0 source and captured execution · LangGraph · official overview

Next: G1: turn the record into a state schema

G1: turn the record into a state schema

> **The schema names fields; the trace annotation selects its merge rule.**

Exact excerpt: Graph_Workflow_Lab.ipynb, G1, lines 1–10. Run the complete cell in context.

```
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

- command and limit describe the trial
- attempts and max_repairs bound retries
- Annotated[list[str], add] preserves events

Highlight the control fields first, then the verdict fields, then the trace annotation. Run the complete G1 cell so the audience also sees the initial validated state and the list-addition example below this excerpt. TypedDict states the intended record structure; initial_state performs the example's explicit input validation. These two mechanisms should not be confused. The trace is a sequence of recorded events, not an internal chain of thought and not a cryptographically authenticated audit log. Ask participants to predict the number of trace entries before revealing the answer. This is the same replace-versus-append idea they saw conceptually. Transition: the first node uses the command to calculate a value, and the next node compares it with the limit.

> **Discussion**
>
> After simulate and verify append one event each, how many remain?

Expected observation: Two events; the list-addition reducer keeps both.

G1 · exact notebook code · Python · TypedDict

Next: G2: a node returns only its updates

G2: a node returns only its updates

> **The calculation and the acceptance test are inspectable functions.**

Exact excerpt: Graph_Workflow_Lab.ipynb, G2, lines 1–7. Run the complete cell in context.

```
def simulate(s):
    value = 0.4 + 0.8 * s["command"]
    return {"current": value, "trace": [f"simulate: {value:.3f}"]}

def verify(s):
    ok = s["current"] <= s["limit"] + 1e-12
    return {"passed": ok, "trace": [f"verify: {ok}"]}
```

- simulate calculates the toy response
- verify evaluates the declared inequality
- Both append a visible event

Read the first function as an ordinary Python calculation. It returns current and a trace update rather than reconstructing the entire state record. The next function compares that current with the limit using a small numerical tolerance. The tolerance addresses floating-point comparison in this toy; it is not an engineering margin. With command 0.75, the result is 1.00 and the limit is 0.80, so the verdict is false. That outcome is expected rather than a program error. Ask which returned field the router will inspect. Run the entire G2 cell, including the repair and terminal-node definitions that follow this excerpt. Transition: inspect the four lines that convert the verdict and budget into a route.

> **Discussion**
>
> For u = 0.75 and L = 0.80, does verification pass?

Expected observation: i = 1.00, so the first verification fails.

G2 · exact simulate and verify functions

Next: G2: the route is four readable lines

G2: the route is four readable lines

> **A successful test reports; a failed test repairs only while budget remains.**

Exact excerpt: Graph_Workflow_Lab.ipynb, G2, lines 19–22. Run the complete cell in context.

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["max_repairs"] else "hold"
```

- Read passed first
- Otherwise compare attempts with max_repairs
- Return one permitted node label

Map each code branch to the earlier route diagram. The successful branch returns report. The failed branch compares the completed repair count with the maximum and returns either repair or hold. Because the same functions will be supplied to the direct-Python baseline, any numerical difference would indicate an implementation issue rather than a more intelligent graph. Ask what happens after one repair when the maximum is one and the verification still fails. The answer is hold, since equality means the budget is exhausted. A recursion limit also appears at invocation, but that runtime limit is not the application repair policy. Transition: node functions and route labels still need connections before they form an executable graph.

G2 · exact conditional router

Next: G3: wire the functions into the diagram

G3: wire the functions into the diagram

> **Edges permit execution paths; compilation does not prove the engineering assumptions.**

Operations discussed: START, Simulate, Verify, Report / repair / hold.

Exact excerpt: Graph_Workflow_Lab.ipynb, G3, lines 7–14. Run the complete cell in context.

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify", route,
        {"report": "report", "repair": "repair", "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=checkpointer)
```

- START enters simulation
- Verification selects report, repair, or hold
- Repair returns to simulation

Run G3 and confirm the compiled node list. Highlight the fixed edge from simulate to verify, the conditional edge that calls route, and the edge returning repair to simulate. The mapping translates a returned label into a permitted successor. The report and hold nodes terminate this version of the execution. Compilation checks structural requirements of the graph; it is not a proof that the chosen response law describes a feeder or that the repair rule finds an optimum. This keeps the software lesson separate from the later numerical science. Ask a participant to point to the line that makes the graph cyclic. Transition: now test three starting states whose outcomes the class can calculate by hand.

G3 · exact graph wiring · LangGraph · graph construction

Next: G4: predict, run, and trace three outcomes

SLIDE 35 · F34

G4: predict, run, and trace three outcomes

> **A legitimate workflow can finish ready or hold.**

Initial u	Limit L	Budget K	Expected result
0.25	0.80	2	Ready · i = 0.60
0.75	0.80	2	One repair · i = 0.80
0.90	0.50	1	One repair, hold · i = 0.92

- Run G4 unchanged
- Trace the repair case event by event
- Try a new budget in a separate scratch cell

Give participants time to predict before running. The first case passes immediately. The second begins at current 1.00, reduces the command from 0.75 to 0.50, and then reaches current 0.80. The third begins at command 0.90, repairs to 0.65, and still exceeds its 0.50 limit with current 0.92. Its single repair is exhausted, so hold is correct. Run G4 unchanged because its assertions describe these original three cases. For a new limit or repair budget, invoke the graph in a separate scratch cell. A useful extension gives the third case four repairs: it reaches command zero and current 0.40. Transition: compare the graph's outputs with the same functions in an ordinary Python supervisor.

> **Experiment / prediction**
>
> Which case reaches the repair node but still must hold?

Expected observation: The third: u becomes 0.65, giving i = 0.92 > 0.50.

G4 · executed three-case experiment

Next: G4: the conventional baseline produces the same result

G4: the conventional baseline produces the same result

> **The graph matches all three commands, outcomes, and traces.**

Exact excerpt: Graph_Workflow_Lab.ipynb, G4, lines 15–19. Run the complete cell in context.

```
from src.graph_workflow_lab import fixed_baseline
for name, args in cases.items():
    baseline = fixed_baseline(initial_state(*args), simulate, verify, repair, route, ready, hold)
    assert summary(baseline) == summary(results[name])
print("Direct Python baseline matches every command, outcome and trace: 3/3.")
```

- Same simulation and verification functions
- Same repair rule and stopping policy
- 3/3 outcomes and event histories match

Make the null numerical result a central teaching outcome. Both implementations receive the same initial state and the same node functions. The comparison asserts equality of the summary, including the command, status, and trace. Therefore the graph should not be credited with finding a better action. Its value here is making execution structure and later checkpoint mechanisms explicit. The notebook prints elapsed times from one local execution, but those values are not a statistically controlled performance benchmark. Differences at this scale depend on runtime and measurement overhead. Ask which separate experiment would support an inspectability or recovery claim. Transition: introduce the graph feature that the direct calculation alone does not yet demonstrate, a real pause with resumable review state.

> **Discussion**
>
> What did the graph add in this experiment?

Expected observation: An explicit execution structure and reusable pause/state mechanisms; it did not improve the toy command.

G4 · direct-Python baseline assertion

Next: G5: stop at the pending review node

G5: stop at the pending review node

> **Before resumption, the report node is pending and acknowledgment is false.**

Exact excerpt: Graph_Workflow_Lab.ipynb, G5, lines 11–17. Run the complete cell in context.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
```

- Build with a checkpointer
- Invoke the graph with a thread identifier
- Inspect the snapshot and interrupt payload

The full G5 cell both pauses and supplies a scripted response, so explain how to observe the intermediate state. For a live teaching pause, execute lines 1 through 17 first, retaining the function definition above this displayed excerpt. Inspect the printed pending node, which is report, and the false acknowledgment flag. The interrupt payload identifies the command and the invented model scope. In the captured pause, the stored status remains TOY_NEW; it should not be mislabeled TOY_READY merely because the numerical gate passed. The checkpoint exists in this kernel's memory. Participants can now see that a verified numerical result and an acknowledged recommendation are different states. Transition: resume the same execution with an explicit response tied to its recommendation identifier.

G5 · captured pre-resume snapshot · LangGraph · persistence

Next: G5: resume with the matching readback

G5: resume with the matching readback

> **The scripted acknowledgment changes status without dispatching equipment.**

Exact excerpt: Graph_Workflow_Lab.ipynb, G5, lines 18–21. Run the complete cell in context.

```
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
print("After simulated readback:", summary(resumed))
assert resumed["acknowledged"] and review_app.get_state(config).next == ()
```

- Use the same execution configuration
- Approve the same recommendation identifier
- Inspect TOY_ACKNOWLEDGED and the final trace

Execute the remaining lines after the pause, or inspect both printed stages if participants ran the full cell. The response is explicitly simulated and generated by the notebook, not provided by an authenticated operator. Its identifier must match the recommendation constructed from the saved state. The resumed graph records acknowledgment and has no pending node. To discuss rejection, point to the human_review function: it requires an approval value of true and the correct identifier; otherwise it returns hold. Avoid resuming an already completed execution for this experiment; create a fresh review run before trying a different response. No equipment endpoint is present. Transition: even a correctly matched review becomes insufficient if its underlying request or candidate has changed.

> **Discussion**
>
> What should a declined or wrong-identifier response produce?

Expected observation: TOY_HOLD, with acknowledgment false.

G5 · simulated readback and final assertion · LangGraph · interrupt/resume

Next: L1: change the request; reject the old evidence

L1: change the request; reject the old evidence

> **Evidence consistency must be checked again after a request or command changes.**

Comparison	Binding
Same request and command	True
Changed load, old evidence	False
Changed command, old evidence	False

Exact excerpt: Classroom_Labs.ipynb, L1, lines 1–5. Run the complete cell in context.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

- Run L0, then L1
- Predict three binding results
- Explain what matching hashes do not establish

Allow four minutes for setup, prediction, execution, and explanation. The initial request uses load scale 1.00, rating fraction 0.80, and revision one. L1 changes the load to 1.10 and revision to two, then separately tests a changed command. The expected binding results are true, false, and false. The payload is deliberately marked example_only; this is an evidence-identity experiment, not a fresh feeder solve. A matching hash establishes consistency among these recorded inputs and the bound result. It does not establish source authenticity, correct intent, or physical feasibility. Fixed supervisors can use the same guard, so the test demonstrates an implementable workflow property without claiming an exclusive LLM benefit. Transition: now choose numerical methods that can produce a meaningful physical candidate.

L0–L1 · exact binding experiment · Tutorial binding implementation

Next: Model-based and model-free methods enter the same workflow

Model-based and model-free methods enter the same workflow

> **MPC and SAC are candidate generators that still face a common verifier.**

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k) \qquad\qquad u_t = \pi_\theta(o_t)$$

H, k	Planning horizon and predicted step	x_k, u_k, ℓ	State, control, and declared stage cost
o_t, π_θ	Observation and learned policy with parameters θ	t	current decision time

- Model-based: optimize predicted behavior
- Model-free: learn an action rule from experience
- Voltage lab: MPC planning and saved SAC inference
- PPO is discussed as an alternative, not trained here

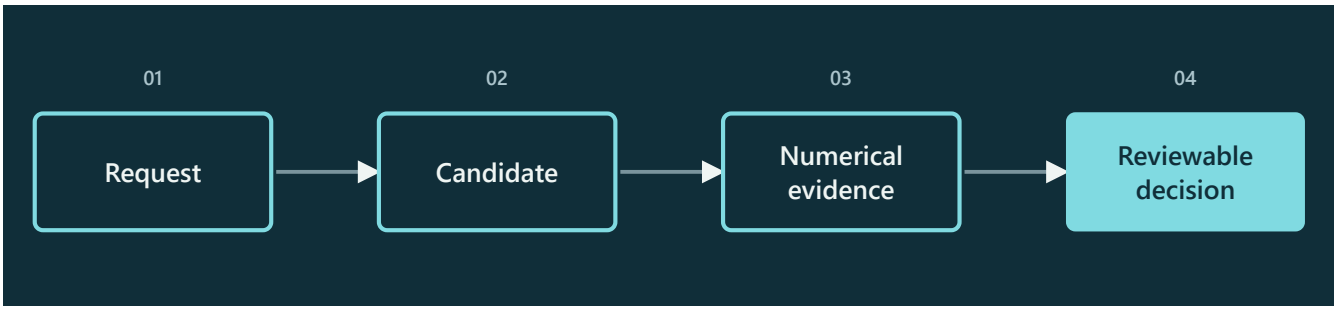
Define the two broad approaches before naming the particular algorithms. Model-based control uses predicted behavior in an optimization problem; the displayed expression omits constraints for clarity, and the voltage lesson supplies them explicitly. Model-free learning trains an action rule through experience and rewards. It may obtain that experience from a simulator, so model-free does not mean no simulator was used. MPC plans over a horizon and applies its first action. SAC supplies continuous control from saved learned weights in class. PPO is a relevant continuous-action alternative but is not trained in this tutorial. Training cost and inference cost are separate. Both proposed commands must be simulated and checked under the same declared limits. Transition: leave the illustrative graph and establish a fair experiment on the selected feeder models.

Rawlings, Mayne & Diehl · MPC · Haarnoja et al. · SAC · ICML 2018 · Schulman et al. · PPO · 2017

Next: The graph is ready; now ask a physical question

The graph is ready; now ask a physical question

> **Compare the same scenario, constraints, budget, and evidence boundary.**



Completed teaching diagram.

- MPC and SAC propose inverter commands
- LinDistFlow and OpenDSS expose model-scope differences
- Llama proposes bounded follow-up work
- Measure objective, rejected checks, and added time

Recap the transferable mechanisms in one sentence: state records the case, tools return evidence, routes handle success and repair, and review records a response to a particular recommendation. The G lab established those mechanisms but did not establish grid-control performance. The engineering notebook now switches to actual numerical models and recorded model responses. Keep the comparison fair by fixing the scenario, permitted controls, declared constraints, objective, and candidate budget wherever a paired comparison is claimed. Report changes in model scope explicitly rather than treating them as unexplained error. The next section introduces the electrical meaning of active and reactive control before displaying the optimization. Transition: begin voltage support with the feeder, the action variables, and the operating limits an operator actually cares about.

> **Discussion**
>
> What can a favorable explanation never substitute for?

Expected observation: A fresh calculation and passing checks for the actual candidate and declared model scope.

L2–L5 · voltage experiments · Matched evaluation principle · PowerAgentBench-SS

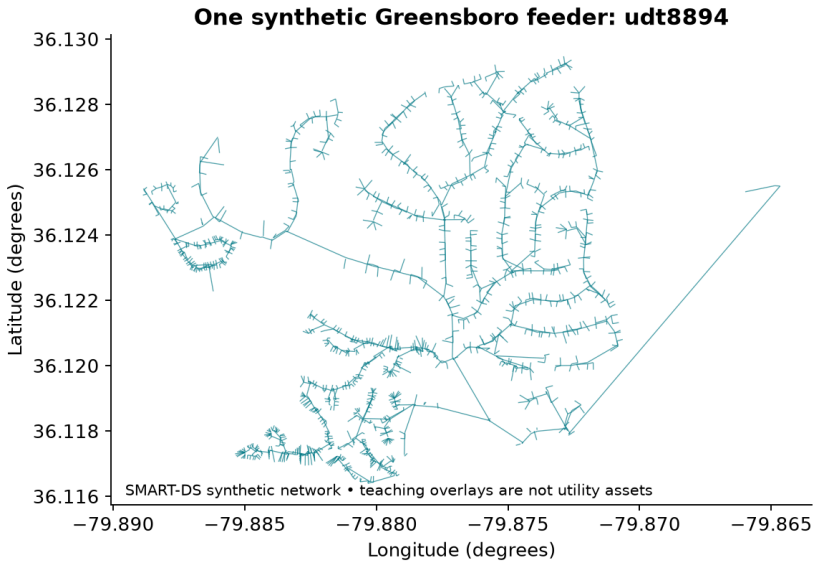
Next: One feeder. Two controls. One checked recommendation.

Voltage support

SLIDE 42 · V01

One feeder. Two controls. One checked recommendation.

> **Start with an operating request whose physical scope is explicit.**



SMART-DS synthetic Greensboro feeder uhs20_1247–udt8894. One selected feeder; no live utility deployment.

- Use available PV
- Respect voltage and current limits
- Keep the operator in the decision loop

The voltage chapter uses one SMART-DS synthetic Greensboro feeder, identified as uhs20_1247–udt8894. It is not a model of the whole city or a live utility. Ask learners which information they still need before advising an operator: device availability, thermal ratings, voltage bounds, model coverage and permitted actions. The map establishes geographic context; it does not establish operational validity. Our numerical experiment uses a fleet command with two continuous variables. All recommendations remain advisory simulation outputs. This chapter moves from a familiar control problem to the reason for explicit agent handoffs: a recommendation should carry the evidence and assumptions that justify it.

> **Discussion**
>
> What must the operator know before accepting a new inverter setting?

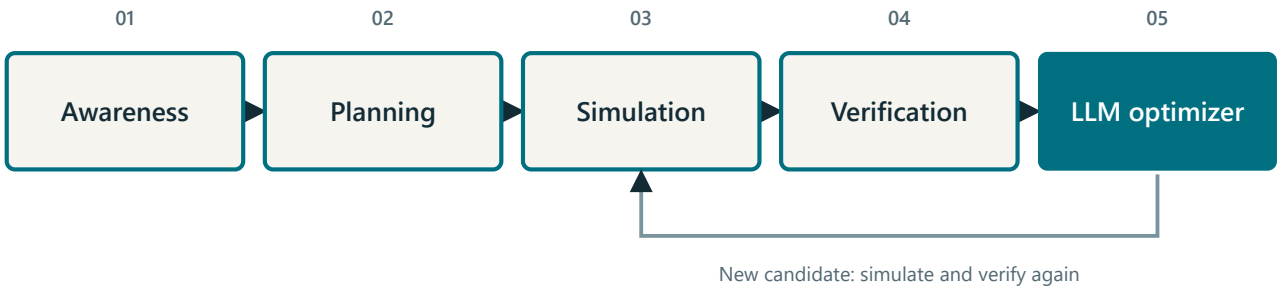
SMART-DS · synthetic distribution models · Executed tutorial · Classroom Labs

Next: Follow the evidence through all five stages

SLIDE 43 · V02

Follow the evidence through all five stages

> **Every stage returns an inspectable result, even without an LLM call.**



Completed teaching diagram.

- Awareness → validated request
- Planning → MPC and SAC proposals
- Simulation and verification → checked evidence
- LLM optimizer → bounded next trial

Animate the stage path in order and keep the final loop visible. Awareness validates the operating request. Planning obtains model-based MPC and model-free SAC proposals. Simulation evaluates their physical consequences with a selected model. Verification checks numerical limits and binds the evidence to the request and candidate. The LLM optimizer can select a permitted refinement; a changed action must return to simulation and verification. A final recommendation is passed to the operator. These responsibilities are distinct from calling a language model five times. Deterministic numerical stages are essential parts of the agentic workflow. Ask participants to predict which stages must repeat if the reactive-power command changes.

> **Discussion**
>
> Which stages repeat when the command changes?

Rawlings, Mayne & Diehl · MPC · Haarnoja et al. · SAC (ICML 2018) · Executed tutorial · Classroom Labs

Next: Awareness: make assumptions machine-checkable

Awareness: make assumptions machine-checkable

> **A useful request identifies what must be true before planning.**

Exact excerpt: Classroom_Labs.ipynb, L1, lines 1–5. Run the complete cell in context.

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

- Task: voltage support
- Load multiplier and current derating
- Model scope and request revision
- Output: validated contract or rejection

Connect the request object to the earlier tool-contract lesson. The load multiplier describes the scenario; the rating fraction sets a thermal condition; the backend specifies the numerical model. These are engineering inputs, not values that a language model may silently choose. A revision distinguishes a changed operating condition from the previous request. The classroom validator checks required fields and allowed values before numerical work begins. This does not authenticate a sensor or prove that the inputs represent the actual feeder. The returned contract is a clear starting point for later checks. Ask learners to identify one field that would require a new simulation if it changed.

> **Discussion**
>
> Does validating a request prove that its measurements are true?

Expected observation: No. Schema validity, provenance and physical truth are different properties.

Executed tutorial · Classroom Labs

Next: Model-based and model-free methods share an action contract

SLIDE 45 · V04

Model-based and model-free methods share an action contract

> **Compare candidate actions under the same scenario, cost and limits.**

Operations discussed: Observation, MPC or SAC, Candidate (a,b), Fresh physical check.

Method	Decision-time work	Physical gate
MPC	Model + optimization	Required
SAC	Saved actor inference	Required
PPO	Discussed alternative	Required

- MPC: predict with a model, then optimize
- SAC: infer with a trained continuous-action policy
- PPO: another continuous-action option, discussed only
- Both: return the proposed command for checking

Explain model-based and model-free before naming algorithms. MPC uses a prediction model while solving the current decision. SAC learns a policy from interaction and produces an action from an observation at inference time. A model-free policy may still have been trained in a simulator; the overall workflow certainly still uses engineering models for verification. SAC is the trained actor executed here. PPO is a relevant alternative, not a second trained result in this tutorial. The same request and acceptance limits apply to both proposals. A fast policy call is useful only if its physical outcome and total workflow cost are also acceptable. This motivates the common candidate contract.

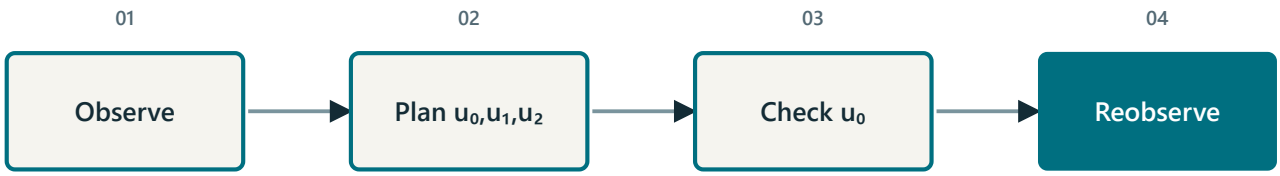
Rawlings, Mayne & Diehl · MPC · Haarnoja et al. · SAC (ICML 2018) · Schulman et al. · PPO (2017)

Next: MPC plans ahead, then uses the first move

SLIDE 46 · V05

MPC plans ahead, then uses the first move

> **A horizon informs the current action; new observations trigger replanning.**



Completed teaching diagram.

- Observe the present state
- Optimize a sequence over H steps
- Test and apply only the first command
- Update the forecast and solve again

Build the timeline one element at a time. The central concept is repeated optimization using new information, rather than executing an entire old plan without revision. The generic MPC framework can contain dynamics and long horizons. This tutorial implements H=3 quasi-static power-flow snapshots, linked by a penalty on command movement. It returns the first proposed control for a fresh check. It does not demonstrate transient stability or a complete closed-loop field rollout. Ask learners why a new forecast could change the later controls and why only the first control is considered now. The next slide formalizes these ideas without hiding the difference between general MPC and the implemented approximation.

Rawlings, Mayne & Diehl · MPC

Next: Read the MPC objective from left to right

Read the MPC objective from left to right

> **Prediction and constraints are part of the optimization problem.**

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \gamma^k \ell(x_k, u_k) \quad \text{s.t.} \quad x_{k+1} = f(x_k, u_k, d_k), \quad g(x_k, u_k) \le 0$$

x_k	Predicted state at step k	u_k, d_k	Control and forecast demand
H, γ	Horizon and discount; tutorial H=3, γ=0.98	ℓ, f, g	Stage cost, prediction model, constraint function

- Minimize discounted stage costs
- Enforce the prediction model
- Keep every planned step inside constraints

First read the symbols aloud. State x_k summarizes the system at step k, u_k is a proposed control and d_k is forecast demand. The stage cost ell encodes preferences, while g represents constraints that preferences cannot waive. The displayed dynamic relation is the standard general formulation. Our implementation specializes it to algebraic power-flow snapshots, a three-step horizon and movement coupling between controls. The discount factor is 0.98. There is no terminal invariant-set construction or closed-loop stability theorem in this teaching example. Have learners point to the part of the equation that would change if the forecast changes, then to the part that enforces a device rating.

Rawlings, Mayne & Diehl · MPC

Next: Two numbers specify the inverter fleet command

Two numbers specify the inverter fleet command

> **Define units and signs before interpreting any numerical result.**

$$u = (a, b), \qquad p_i = aS_i, \qquad q_i = bS_i$$

u	Two-number fleet command	a, b	Dimensionless active and signed reactive participation
S_i	Inverter i apparent-power rating, kVA	p_i, q_i	Active kW and reactive kvar commands

- a sets active participation
- b sets signed reactive participation
- Negative b means reactive absorption
- Every inverter retains its own rating

The tutorial uses a reduced fleet control rather than thousands of independently optimized inverter setpoints. For inverter i, S_i is the apparent-power rating and the dimensionless pair a,b scales active and reactive commands. Here the available active capability follows the adapter's declared teaching assumption. The negative sign convention for b corresponds to reactive absorption. Learners should not infer the sign from a plot without reading this contract. Ask what happens to each device when a increases by 0.0025. The physical change is proportional to that device's rating. This shared two-number interface lets MPC, SAC and a bounded language-selected refinement propose directly comparable controls.

Executed tutorial · Classroom Labs · Recorded experiment · LLM response selection

Next: Four cost terms express four preferences

Four cost terms express four preferences

> **A cheaper reactive command can still worsen the total outcome.**

$$\ell = (1-a)^2 + 0.05b^2 + 10\,\mathrm{mean}[(V-1.01)^2] + 0.05\|u-u_{\mathrm{prev}}\|_2^2$$

V	Modeled voltage magnitudes, p.u.	u_{prev}	Previous two-number command
ℓ	Dimensionless weighted teaching cost		

- Preserve active capability
- Limit reactive effort
- Keep voltages near 1.01 p.u.
- Avoid abrupt changes from the previous command

Reveal the cost one term at a time and ask learners to name the engineering preference represented by each term. The first term discourages curtailment under the declared availability assumption. The second discourages reactive effort. The third measures mean squared voltage deviation over the modeled bus-phase set. The fourth penalizes command movement. All weights are teaching design choices, not universal operating preferences. Reducing the magnitude of b certainly reduces the reactive-effort term, but can alter voltages, movement and feasibility. The objective is evaluated for the current step in the reported comparisons. A better objective alone establishes neither stability nor a safe recommendation outside the modeled contract.

> **Discussion**
>
> Does reducing reactive effort necessarily reduce the total cost?

Expected observation: No. The voltage and movement terms can change.

Recorded experiment · LLM response selection

Next: Feasibility comes before ranking

SLIDE 50 · V09

Feasibility comes before ranking

> **Neither a low cost nor a majority vote can waive a limit.**

$$V_{\min} \le |V_i| \le V_{\max}, \quad |I_e| \le I_e^{\max}, \quad a^2 + b^2 \le 1, \quad 0 \le a \le 1$$

V_i	Bus-phase voltage, p.u.	$I_e, I_e^{\max}$	Branch current and allowed current, A
a, b	Normalized active and reactive command		

- Network: voltage and current bounds
- Devices: inverter capability and action bounds
- Evidence: finite values and completed solve
- Identity: matching request, command and model

Draw the inverter capability circle while revealing the network bounds. The circle follows from the apparent-power limit after dividing by the squared rating. The planner uses local sensitivities and a conservative capability polygon; the final proposal still returns to a numerical model for checking. Distinguish a solver returning normally, a good cost and a feasible candidate. These are separate observations. Evidence validity also requires matching case, candidate and model identifiers, so results from a previous setting cannot be attached to a new one. Ask whether three language models agreeing can make a branch overcurrent acceptable. The answer locates the engineering gate above preference selection.

> **Discussion**
>
> Can three agreeing LLMs turn an overcurrent into PASS?

Expected observation: No. Voting chooses a trial; numerical constraints determine admissibility.

Rawlings, Mayne & Diehl · MPC · Recorded experiment · LLM response selection

Next: The two backends see different constraint sets

SLIDE 51 · V10

The two backends see different constraint sets

> **Declare model coverage before comparing outcomes.**

Operations discussed: Same command, MV-only check, Full MV/LV check.

Model	Covered network	Meaning of PASS
LinDistFlow3	MV + ideal LV aggregation	Reduced contract
OpenDSS	Supplied MV/LV + transformers	Full snapshot contract

- LinDistFlow3: MV network, ideal aggregated LV demand
- OpenDSS: supplied MV/LV feeder and transformers
- Same action can pass one contract and fail another
- Changed coverage prevents an accuracy-only comparison

Keep two model cards visible while adding their coverage. Python LinDistFlow3 implements a coupled three-phase linear approximation on the MV network. Its ideal aggregation does not expose LV voltages or transformer thermal constraints. OpenDSS solves the supplied unbalanced MV/LV power-flow model. Both are useful for their declared purpose, but their PASS labels answer different questions. This experiment changes both model equations and coverage. It therefore cannot isolate a pure approximation-error effect. Even the voltage term in the objective is aggregated over different modeled sets. Ask learners which additional limits could fail when the same command is sent to the broader model.

Gan & Low · three-phase optimization (2014) · EPRI · OpenDSS documentation · Executed tutorial · Classroom Labs

Next: LinDistFlow3 retains three-phase coupling

LinDistFlow3 retains three-phase coupling

> **The linear relation still contains mutual impedance between phases.**

$$\mathbf{w}_j = \mathbf{w}_i - 2\,\mathrm{Re}\left[\overline{\boldsymbol{B}} \odot \left(Z_{ij}\left(\boldsymbol{\beta} \odot \overline{\mathbf{S}}_{ij}\right)\right)\right]$$

$\mathbf{w}_i, \mathbf{w}_j$	Parent and child squared phase voltages, V²	$Z_{ij}, \mathbf{S}_{ij}$	Coupled impedance Ω; downstream phase powers VA
$\boldsymbol{\beta}$	Nominal phase ratios (1, $e^{-j2\pi/3}$, $e^{-j4\pi/3}$)	$\overline{\cdot}, \odot, \mathrm{Re}$	Conjugate, entrywise product, real part

- Propagate squared voltages down the radial feeder
- Use phase impedance matrices
- Assume nominal phase-angle ratios
- Report losses diagnostically; omit them from branch balance

Start from the parent and child buses, then reveal the voltage-drop term. The equation matches the fixed phase-ratio closure used by the Python adapter. The matrix Z contains mutual phase coupling, so this is not a single-phase equivalent. The beta vector encodes nominal phase relationships, overbars denote complex conjugation, the circle-like multiplication symbol means entrywise multiplication and Re extracts the real component. Branch powers are in VA and squared voltages in volts squared. The imaginary unit in the exponent is distinct from a bus index. Currents use fixed source-reference phasors at 1.03 p.u.; small-loss assumptions are not certified error bounds. The full limitations remain in the handout.

Gan & Low · three-phase optimization (2014) · Executed tutorial · Classroom Labs

Next: SAC balances task reward and exploration

SLIDE 53 · V12

SAC balances task reward and exploration

> **A reward penalty is not a proof of constraint satisfaction.**

$$\max_{\pi} \mathbb{E}\left[\sum_{t\geq 0} \gamma^t \{r(s_t, u_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t))\}\right]$$

s_t, u_t	Observation and continuous action	π, r	Actor policy and task reward
$\mathcal{H}, \alpha$	Policy entropy and exploration weight	γ	Reward discount factor
$\mathbb{E}, t$	Expectation over policy trajectories; discrete training step		

- Actor maps observations to continuous actions
- Reward evaluates the task outcome
- Entropy encourages exploration during learning
- The frozen actor still needs fresh verification

Use this equation to explain the purpose of SAC rather than derive every update rule. The actor is a probability distribution over continuous actions. The task reward expresses operational preferences and penalties, while entropy encourages broader exploration during training. Alpha controls that tradeoff and gamma discounts future rewards. Training and inference are different phases: the classroom loads saved weights and computes a new action for the current observation. It does not retrain the policy. Even a strong reward penalty for violations can leave unsafe actions under distribution shift. This is why the actor is a candidate generator within the workflow, followed by simulation and a hard acceptance gate.

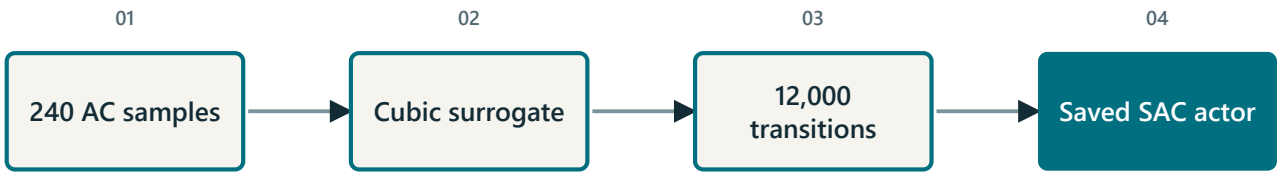
Haarnoja et al. · SAC (ICML 2018) · Schulman et al. · PPO (2017)

Next: Read the training provenance, not only the actor output

SLIDE 54 · V13

Read the training provenance, not only the actor output

> **Training cost and inference cost belong to different measurements.**



Completed teaching diagram.

- 240 AC solves fit a cubic surrogate
- 48 separate samples evaluate the surrogate fit
- 12,000 policy-training transitions use that surrogate
- L2 performs new inference with saved weights

Animate a pipeline from AC samples to surrogate fitting to policy training to the saved actor. The 12,000 transitions are not 12,000 OpenDSS solves. This distinction matters for reproducibility and for reporting computational cost. The fit-validation samples assess the surrogate, not all aspects of policy robustness. New classroom scenarios can alter demand or current-rating conditions, creating a transfer problem for the frozen actor. A quick inference time therefore says little about total development cost or reliability under shift. Ask learners what evidence they would request before trusting the actor beyond this fixture. Possible answers include separate policy test cases, out-of-distribution conditions, constraint failures and an appropriate deterministic baseline.

> **Discussion**
>
> What exactly is being recomputed when we run L2?

Haarnoja et al. · SAC (ICML 2018) · Executed tutorial · Classroom Labs

Next: L2: the code changes a scenario, then recomputes

L2: the code changes a scenario, then recomputes

> **One cell solves MPC, infers SAC and checks both candidates.**

Exact excerpt: Classroom_Labs.ipynb, L2, lines 1–7. Run the complete cell in context.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
timings['L2 fresh MPC + SAC + checks'] = time.perf_counter() - started
print(voltage['mode'])
display(voltage['table'])
```

- Change one scenario parameter
- Call the shared experiment helper
- Time the whole numerical operation
- Inspect the returned comparison table

Walk through the exact first seven lines of L2. LOAD_SCALE and RATING_FRACTION are the two visible scenario controls. The helper performs a fresh MPC calculation, new saved-actor inference and new numerical checks. The timer encloses the helper call, so it measures more than policy inference alone. The returned table is the place to compare feasibility and cost before looking at the plot. Keep the original default outputs as a reference, then make one change at a time. The later L3 response pool belongs to a different archived original context; rerunning L2 does not regenerate those LLM responses. This boundary must stay explicit during the exercise.

> **Discussion**
>
> Which output would show that a faster proposal was unusable?

Executed tutorial · Classroom Labs

Next: Lab: predict, change demand, explain the outcome

Lab: predict, change demand, explain the outcome

> **Change one assumption and support your conclusion with a metric.**

Exact excerpt: Classroom_Labs.ipynb, L2, lines 1–4. Run the complete cell in context.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
```

- Predict before running
- Run L2 at the default load
- Change LOAD_SCALE once and rerun L2
- Record feasibility, objective and the limiting margin

Allow seven minutes and keep this slide visible while participants work. One learner predicts and one operates the notebook, then they exchange roles. First record the default results. Next choose a modest changed load multiplier and predict how PV participation, reactive support or curtailment might respond. Run the full L2 cell and compare the same acceptance metrics. Students should explain a result using a numerical margin rather than a general claim about AI. If setup fails, use the executed notebook output to complete the prediction and interpretation tasks. Do not modify unrelated cells or imply that the archived LLM pool has changed. Finish by collecting one surprising observation from the room.

> **Experiment / prediction**
>
> At higher demand, will PV curtailment necessarily increase?

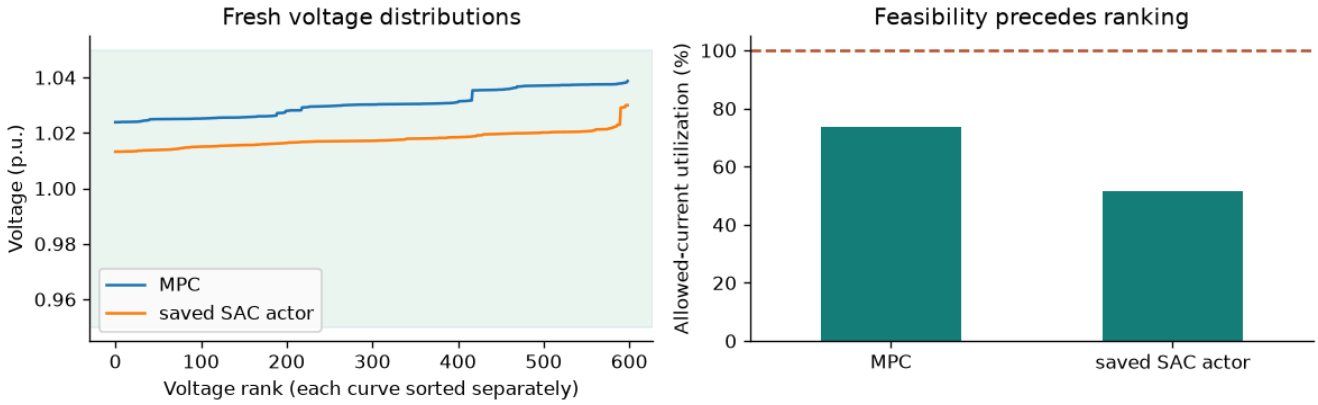
Expected observation: Not necessarily. Local voltage and current constraints can move in different directions.

Executed tutorial · Classroom Labs

Next: Inspect the margins before the winner label

Inspect the margins before the winner label

> **A feasible action can still be much worse on the declared objective.**



Executed L2 default: fresh MPC solve, saved SAC actor inference and reduced-model checks. Sorted curves are voltage ranks.

- MPC objective: 0.031104
- SAC objective: 0.436428
- Both pass the reduced-model gate in this default case
- Voltage curves are separately sorted distributions

Use the actual executed L2 figure. The displayed default costs are rounded for teaching: MPC is approximately 0.031104 and SAC 0.436428, with both feasible under the reduced-model contract. The voltage curves are sorted separately, so matching positions are ranks rather than the same physical bus. The current-utilization plot reports the most heavily used allowed current bound. Ask participants to identify the evidence for feasibility before comparing objective values. A cost gap here is a controller result, not automatically an advantage of the multi-agent architecture. Students with changed load settings should report their own values and avoid mixing them with this original reference output.

> **Discussion**
>
> Does the lower objective identify which stage caused the improvement?

Executed tutorial · Classroom Labs

Next: L3: inspect actual archived model responses

L3: inspect actual archived model responses

> **Recorded model output is evidence to inspect, not a fresh measurement.**

Exact excerpt: Classroom_Labs.ipynb, L3, lines 1–8. Run the complete cell in context.

```
pool = lab.saved_pool('test_01')
display(pd.DataFrame(pool['responses'])[['model_id', 'operation_id', 'reason', 'seconds', 'source']])
display(pd.DataFrame([{'operation': ', '.join(c['operation_ids']),
    'votes': c['vote_count'], 'share': c['vote_share'], 'parameters': c['operation']['parameters']}
    for c in pool['pool']['candidates']]))
print('ARCHIVED ACTUAL LLM; original context:', pool['context']['id'])
print('Distinct physical candidates:', len(pool['pool']['candidates']))
print('Malformed/duplicate votes rejected:', len(pool['pool']['rejected']))
```

- Load original test_01
- Display model, operation, reason and recorded seconds
- Count distinct physical candidates
- Keep this context separate from modified L2

Explain why this cell is reproducible without contacting a local model. It loads an archived response pool from original test_01, displays each exact public response and reports how the structured proposals were canonicalized. The seconds column contains recorded call durations, not the time required to print the table. The original test context is not identical to L2's default rating fraction, and certainly not to a student-modified L2 scenario. This cell therefore teaches inspection and evidence attribution. It does not supply new context-matched recommendations for the latest classroom perturbation. Ask learners to find a claim in a reason that is supported by the numeric evidence and one that still requires testing.

Executed tutorial · Classroom Labs · Recorded experiment · LLM response selection

Next: Three replies produce two different physical trials

Three replies produce two different physical trials

> **A persuasive explanation can contain an unsupported engineering claim.**

Recorded model	Selected operation	Call time
Llama 3.1 8B	Increase a by 0.0025	1.867 s
Llama 3.2 3B	Reduce |b| by 10%	0.905 s
Qwen 2.5 7B	Reduce |b| by 10%	1.844 s

- Llama 3.1 8B: increase a by 0.0025 · 1.867 s
- Llama 3.2 3B: reduce |b| by 10% · 0.905 s
- Qwen 2.5 7B: reduce |b| by 10% · 1.844 s
- Steady-state tests do not establish voltage stability

These times and choices come from the recorded response-selection pilot. Llama 3.1 suggested increasing active participation to reduce curtailment. Llama 3.2 described reducing reactive effort and also mentioned reducing curtailment, a causal connection that its text alone does not establish. Qwen used the phrase voltage stability, which is unsupported by these steady-state tests. Present the excerpts as public rationales, not access to private model reasoning. Llama names the model family; Ollama is the local runtime that served it. Have the audience annotate the difference between a permitted operation and a justified physical claim. The next step converts prose-backed choices into reproducible physical candidates.

> **Discussion**
>
> Qwen wrote “may help improve voltage stability.” What evidence supports that?

Expected observation: These steady-state tests do not establish voltage stability.

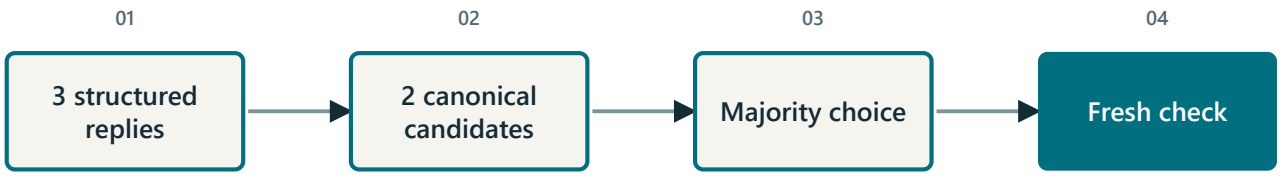
Recorded experiment · LLM response selection

Next: Vote on a canonical operation

SLIDE 60 · V19

Vote on a canonical operation

> **Voting selects the next trial; verification controls whether it is retained.**



Completed teaching diagram.

- Parse structured tool and parameter fields
- Deduplicate identical physical commands
- Count votes with a fixed tie rule
- Simulate and verify the selected trial

Animate three response cards merging into two physical-candidate cards. Two different descriptions of the same command must not count as two distinct control actions. Canonical identifiers depend on the tool and parameters rather than rhetorical style. In original test_01, two models propose the same reactive reduction and one proposes a PV increase, so the majority chooses the reactive trial. A predetermined hash tie-break makes ties reproducible; it is not an engineering preference. Agreement is also not a calibrated safety probability because models can share mistaken assumptions. A changed command returns to the numerical stages. Ask what should happen if the majority-selected trial fails a current constraint.

> **Discussion**
>
> What happens if the majority trial fails?

Expected observation: Reject that trial; retain a valid incumbent or escalate.

Recorded experiment · LLM response selection

Next: LinUCB learns which operation is worth testing

LinUCB learns which operation is worth testing

> **The learner chooses among generated operations, not which LLM to call.**

$$q(a \mid s) = \theta^\top \phi(s,a) + \beta\sqrt{\phi(s,a)^\top A^{-1}\phi(s,a)}, \qquad a^* = \arg\max_a q(a \mid s)$$

s, a	Evidence context and canonical operation; a is not inverter participation here	ϕ, θ	Context-operation features and learned coefficients
A, β	Feature-information matrix and exploration weight	q, a^*	Training selection score and highest-scoring permitted operation

- Features combine context and candidate operation
- Training score includes an exploration bonus
- Feedback updates the linear reward model
- Frozen evaluation uses predicted reward only

Explain select, observe and update before revealing the formula. The feature vector represents the known context and a canonical operation, including vote-related information. Theta stores learned reward coefficients; A stores regularized feature information. During training, beta controls an uncertainty bonus. After a trial, selected feedback updates the reward model. The classroom evaluation freezes these parameters and uses predicted reward only, with no test updates. All candidate LLM responses have already been generated, so their call costs are paid by every pool selector. This is a contextual bandit, a simple reinforcement-learning setting, rather than training the language models or adapting a full sequential control policy online.

Li et al. · LinUCB (WWW 2010) · Recorded experiment · LLM response selection

Next: Learning feedback must follow the physical gate

Learning feedback must follow the physical gate

> **A predicted reward cannot override a failed physical check.**

$$r = \begin{cases} \min\left(1, \dfrac{\max(0, J_{\text{inc}} - J_{\text{trial}})}{0.01}\right) - 0.01, & \text{PASS} \\ -1.01, & \text{rejected} \end{cases}$$

r	Post-trial learning feedback	$J_{\text{inc}}, J_{\text{trial}}$	Incumbent and trial current-step costs

- 24 training, 8 validation, 12 test contexts
- Five training orders; selected feedback only
- Frozen validation and test: zero updates
- The 0.01 trial charge is not seconds

Read the reward in two branches. A passing trial can earn a bounded improvement reward relative to the incumbent cost, with a fixed trial charge. A rejected trial receives a negative reward. The 0.01 scaling and charge are design choices; the charge does not measure latency. Training consumes selected outcomes, although the audit records contain other outcomes for comparisons. Validation and test are frozen, avoiding feedback leakage. Five training orders reuse the same training contexts and are not five independent datasets. Ask learners how changing reward scaling might change selection without changing any electrical operating limit. This separates learning preferences from the hard verification contract.

Li et al. · LinUCB (WWW 2010) · Recorded experiment · LLM response selection

Next: L4: test the frozen choice, not the predicted reward

L4: test the frozen choice, not the predicted reward

> **The selected action must earn a new numerical result.**

Exact excerpt: Classroom_Labs.ipynb, L4, lines 1–8. Run the complete cell in context.

```
decision = pool['selection']
print('Frozen:', pool['frozen'], '| selected-feedback training updates:', pool['training_updates'])
print('Bandit operation:', decision.operation_id, '| predicted reward:', round(decision.predicted_reward,
4))
print('Same canonical choice as majority:', decision.candidate_id == pool['vote_id'])
started = time.perf_counter()
selected_trials = lab.trial_saved_selection(pool)
timings['L4 new verification of archived choices'] = time.perf_counter() - started
display(selected_trials['table'])
```

- Run L4
- Compare the learner choice with the majority vote
- Inspect fresh feasibility and cost
- Explain what happens after a failed trial

Read the exact L4 code, then give participants three minutes. The stored selector chooses an operation using its frozen parameters. The numerical helper evaluates the selected trial again under the archived original context. The displayed predicted reward is a learned estimate, while the returned trial table contains newly calculated outcomes. Participants should check whether the canonical candidate matches the majority choice and whether it passes the engineering gate. In this recorded test, the learner and majority agree on reactive reduction. Do not subtract this cost from a modified L2 cost, because those are different contexts. Ask which outcome controls the recommendation if a predicted improvement conflicts with failed verification.

> **Experiment / prediction**
>
> Predicted reward is positive, but the current limit fails. Accept?

Expected observation: Reject. The gate takes precedence over reward prediction.

Executed tutorial · Classroom Labs · Recorded experiment · LLM response selection

Next: A useful experiment can find no learned advantage

A useful experiment can find no learned advantage

> **Compare extra agents with a strong, inexpensive deterministic supervisor.**

MV held-out method	Mean J	Full-feeder AC PASS
Fixed refinement	0.03400656	0 / 12
Majority vote	0.03398479	0 / 12
Frozen learner	0.03398479	0 / 12

- Fixed refinement mean J: 0.03400656
- Vote and learned selector mean J: 0.03398479
- Vote and learner tie on every held-out case
- A posthoc fixed Q-reduction rule reproduces their choices

Present this result without framing a tiny cost change as general superiority. The majority vote and frozen learner tie on all twelve held-out MV cases. A simple posthoc rule that always selects the reactive reduction reproduces those choices without language-model calls. Because that rule was identified after inspecting results, label it a diagnostic rather than a predeclared baseline. The experiment demonstrates that bounded proposals, fresh verification and reproducible traces can be implemented. It does not demonstrate that additional LLMs or a learned selector are necessary for better control. Ask what new test set or task distribution would make the learner's contribution more convincing than the simpler supervisor.

> **Discussion**
>
> Which ablation would test whether language-model calls are necessary?

Recorded experiment · LLM response selection

Next: Account for the full decision path

Account for the full decision path

> **Fast actor inference does not imply a fast end-to-end recommendation.**

MV held-out method	Accounted serial time
Fixed refinement	3.327 s
Frozen learner with response pool	8.119 s

- Fixed-refinement accounted serial time: 3.327 s
- Learned-selection accounted serial time: 8.119 s
- All pool selectors pay for all three model calls
- Separate setup, inference, simulation and verification

The two reported times are reconstructed serial accounting from actual recorded components in the held-out pilot. They are not a guaranteed deployment response time and should not be confused with a new classroom timing measurement. Pool-based selectors share the cost of generating the three responses. The learned selector's small scoring operation is only one component of that total. Cold model loading, prompt processing, token generation and numerical checking can have very different scaling. Ask learners to draw the boundary around the quantity they want to compare. The engineering question is whether any incremental benefit justifies the additional decision latency under the intended operator workflow.

> **Discussion**
>
> What would change if the local model had to load from disk?

Recorded experiment · LLM response selection · Ollama · chat API and timing fields

Next: L5: audit the same action, then replan separately

SLIDE 66 · V25 · L5

L5: audit the same action, then replan separately

> **Cross-model auditing and replanning answer different questions.**

Exact excerpt: Classroom_Labs.ipynb, L5, lines 1–6. Run the complete cell in context.

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=RUN_OPENDSS)
timings['L5 selected model scope checks'] = time.perf_counter() - started
display(scope_audit['table'])
print(scope_audit['optional_mode'])
print('An MV-only PASS cannot acknowledge a request requiring full MV/LV coverage.')
```

- Default: fresh MV check
- Optional: same-action full-feeder OpenDSS audit
- Optional: separate MPC solve using the AC backend
- Keep the frozen selector unchanged

Read the helper call before enabling its optional path. The same-action audit asks whether an existing recommendation survives a broader physical model. The separate AC MPC solve asks what command a planner using that broader model would propose. Mixing those questions can make model coverage look like an LLM improvement. The default classroom execution performs the MV check; enabling RUN_OPENDSS in L0 requests the fresh AC audit and separate AC replanning in L5. The saved learner remains frozen. OpenDSS computations should be performed sequentially because the local engine has shared mutable state. Compare the status, limiting quantities and model scope rather than treating cross-model costs as an exact solver ranking.

Executed tutorial · Classroom Labs · EPRI · OpenDSS documentation

Next: Broader coverage reveals a missing constraint boundary

Broader coverage reveals a missing constraint boundary

> **The reduced-model PASS does not satisfy a full-feeder request.**

Final learned-workflow actions	Selected-model gate	Full-feeder AC audit
MV planning	12 / 12	0 / 12
OpenDSS planning	12 / 12	12 / 12

- MV-planned actions: 12/12 pass the MV gate
- Same actions: 0/12 pass the full-feeder AC audit
- AC-planned final actions: 12/12 pass their AC gate
- Human operating approval remains pending

The table summarizes the separately recorded twelve paired scenarios, not the output of one L5 cell. The reduced model accepts all twelve final actions under its own scope, yet their same-action full-feeder audits fail. Planning with the AC backend returns actions that pass the full-feeder snapshot contract in these scenarios. The result is caused by the broader model and constraint coverage, with the selector still frozen. It cannot be attributed to more convincing Llama explanations. Within the AC pilot, the learned selector also does not establish superiority over fixed or uniform selection. Ask learners how a responsible operator packet should describe an MV-only recommendation when full MV/LV coverage was requested.

> **Discussion**
>
> Which component supplied the extra physical information?

Expected observation: The broader numerical model and its acceptance contract.

Recorded experiment · LLM response selection · EPRI · OpenDSS documentation

Next: Verification returns reasons, not just a green badge

SLIDE 68 · V27

Verification returns reasons, not just a green badge

> **A rejected recommendation should explain the failing contract.**

- Recompute the exact selected action
- Check voltage, current and capability limits
- Bind results to the request and candidate
- Return PASS, rejection reasons and model scope

Return to verification after the cross-model result has made its purpose concrete. A bare boolean cannot explain whether a command failed because of overcurrent, incomplete data, wrong scope or stale evidence. The verifier should retain the numerical metrics, checked bounds and request-candidate identity alongside its decision. The classroom uses hash binding as a consistency mechanism, not as sensor authentication or a proof of truthful computation. The exact selected action receives a final check before handoff. Ask learners what changes when an operator edits only one setpoint: the old evidence is no longer the evidence for that command. This motivates a fresh check rather than simply updating the displayed label.

> **Discussion**
>
> Which fields would you want beside a rejected recommendation?

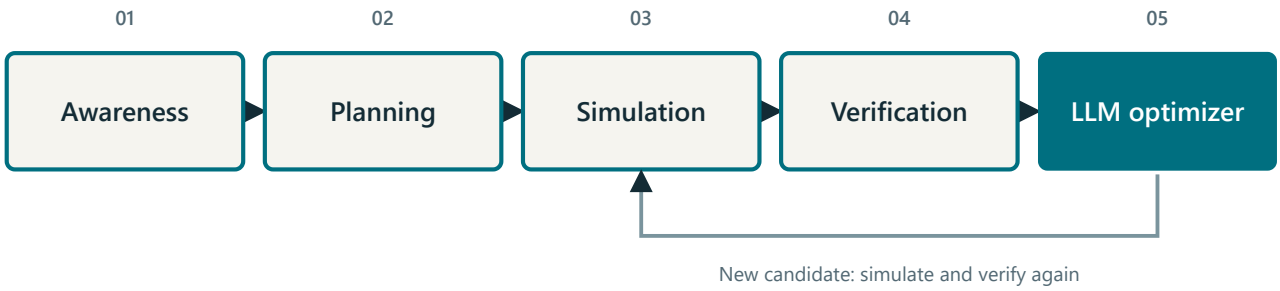
Executed tutorial · Classroom Labs · Recorded experiment · LLM response selection

Next: Voltage support: keep the benefit claim precise

SLIDE 69 · V28

Voltage support: keep the benefit claim precise

> **The workflow exposes assumptions and failures; better control must be measured.**



Completed teaching diagram.

- Awareness: validated scenario
- Planning: MPC and SAC candidate controls
- Simulation and verification: scope-specific physical evidence
- Optimizer and operator: bounded refinement, review and readback

Close the voltage chapter with the complete path and the actual findings. The numerical controllers supplied proposed settings. The language models supplied bounded refinement choices and public explanations. Voting and frozen learned selection tied in the held-out MV pilot. Broader-model checking revealed that the reduced-model recommendations did not meet the requested full-feeder scope. These are valuable workflow lessons without a claim of universal agentic control superiority. The operator must see the command, scenario, evidence, failed checks, remaining assumptions and required scope. Readback remains advisory and does not dispatch equipment. Invite one participant to name a demonstrated benefit and another to name a limitation, then take the scheduled ten-minute break before restoration.

> **Discussion**
>
> Name one demonstrated workflow benefit and one unproven performance claim.

Executed tutorial · Classroom Labs · Recorded experiment · LLM response selection

Next: Ten-minute break

Pause

SLIDE 70 · BREAK

Ten-minute break

> **Next we change the decision from continuous inverter control to discrete restoration.**

- Save the voltage experiment.
- Return with one model-mismatch question.

Take the scheduled ten-minute break. Participants should save their notebook and record whether the LinDistFlow candidate passed each model. At the restart, ask one participant to explain why the same physical action can pass an aggregated model and fail a more detailed model. This prepares the distinction between proposing a candidate and establishing its admissibility. The next chapter introduces discrete restoration decisions and a priority-weighted objective.

Classroom_Labs L2–L5

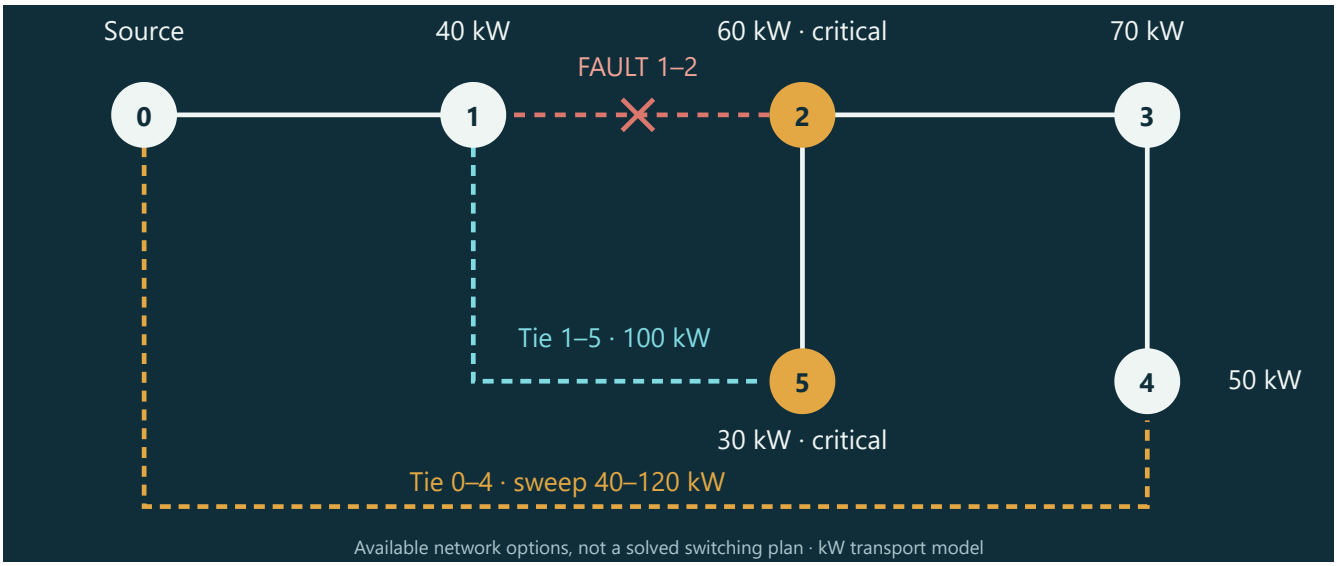
Next: After the fault, decide what can be reconnected

Restoration

SLIDE 71 · R01

After the fault, decide what can be reconnected

> **Restoration chooses topology and load pickup under operating constraints.**



Completed teaching diagram. Exact six-node classroom transport graph; dashed ties are options, not a solved switching plan.

- One faulted edge is unavailable
- Two possible ties reconnect downstream buses
- Critical and ordinary loads compete for capacity

Resume after the break with the six-node restoration drawing. This transparent graph is a separate teaching fixture rather than a reduced claim about the Greensboro feeder. Node zero is the source; edge 1–2 is faulted and unavailable. The original loads are 40, 60, 70, 50 and 30 kW at nodes one through five, with nodes two and five assigned critical priority. One optional tie has a 100 kW capacity; the other has the capacity swept in the exercise. Reveal the ties only after participants locate the fault. Ask which loads might be served without assuming every energized bus must pick up its own load.

> **Discussion**
>
> Can an energized bus transmit power while its local load stays disconnected?

Expected observation: Yes. Bus energization and load pickup are separate decisions.

Chen et al. · restoration optimization (2018) · Wang et al. · radiality constraints (2020)

Next: Three binary decisions describe one restoration plan

Three binary decisions describe one restoration plan

> **Separate pickup, energization and switch closure.**

s_i	Binary load pickup	y_i	Binary energized bus
z_e	Binary closed edge		

- s_i: pick up load i
- y_i: energize bus i
- z_e: close edge e
- A fault exclusion is a hard constraint

Use the same network drawing and highlight a transit bus. A bus may be energized to carry power to another load while its own demand is not picked up. This requires separate binary variables s and y. A line is closed only if its endpoint buses are energized, and the faulted edge is forced open. These logical distinctions prevent an apparently good service total from describing an impossible path. Awareness supplies the isolated fault, permitted ties, capacities and documented priorities. It does not invent permission to operate equipment. Ask learners to translate the transit-bus example into s_i=0 and y_i=1 before revealing the objective on the next slide.

Chen et al. · restoration optimization (2018) · Wang et al. · radiality constraints (2020)

Next: Choose a service objective that matches the instruction

Choose a service objective that matches the instruction

> **Priority weights express policy; they are not an LLM discovery.**

$$\max \sum_i w_i d_i s_i, \qquad s_i \le y_i, \qquad z_{ij} \le y_i,\ z_{ij} \le y_j, \qquad z_{\text{fault}} = 0$$

d_i	Load demand, kW	w_i	Priority weight; critical 10, otherwise 1
s_i, y_i, z_{ij}	Binary pickup, energized bus and closed edge		

- Maximize weighted served demand
- Critical weights are 10; other weights are 1
- Use unit weights for the total-service baseline
- Report both critical and total served kW

Reveal demand, priority and pickup separately in the objective. Demand d_i is nominal active power, w_i is a documented service priority and s_i decides whether that demand is picked up. Assigning a weight of ten to critical loads changes the optimization policy; an improvement in critical service is therefore not evidence that a language model outperformed a numerical optimizer. A finite weight also does not guarantee lexicographic critical-first service for every possible network. A strict priority requirement should be implemented through staged optimization or a verified primary-objective constraint. The notebook reports both total and critical service so that the audience can see the tradeoff rather than only a single weighted score.

Chen et al. · restoration optimization (2018) · Wang et al. · radiality constraints (2020)

Next: Power must reach every served load

Power must reach every served load

> **An attractive pickup list still needs a feasible source path.**

$$\sum_{e} A_{ie} P_e = d_i s_i \quad (i \neq 0), \qquad -\overline{P}_e z_e \leq P_e \leq \overline{P}_e z_e$$

A_{ie}	Signed node-edge incidence entry	P_e	Active branch flow, kW
$\overline{P}_e$	Branch capacity, kW	$i = 0$	Source bus

- Balance active power at each non-source bus
- Limit flow by the closed edge capacity
- An open edge carries no power
- The source supplies the served demand

Walk through one path from source zero to a picked-up load. The incidence matrix assigns a sign to each branch at each bus. The balance equation ensures that the net incoming active power equals served demand. The capacity condition forces zero flow on an open edge and limits the magnitude of flow on a closed edge. Signed P allows either direction according to the chosen orientation. This is an active-power transport formulation, not a full AC power-flow model. It omits voltage and reactive power so that the discrete restoration logic is visible. Before a physical switching recommendation, those omitted constraints and equipment permissions would require their own checks.

Chen et al. · restoration optimization (2018) · Wang et al. · radiality constraints (2020)

Next: Radiality needs connectivity as well as an edge count

Radiality needs connectivity as well as an edge count

> **A tree equation alone does not exclude a disconnected island.**

$$\sum_e A_{ie} f_e = y_i \ (i \neq 0), \qquad |f_e| \leq 5 z_e, \qquad \sum_e z_e = \sum_i y_i - 1$$

f_e	Fictitious connectivity flow, not electrical power	5	Maximum number of energized non-source buses
$\sum z_e = \sum y_i - 1$	Tree edge count, used together with connectivity		

- Send one fictitious unit to each energized non-source bus
- Allow that commodity only on closed edges
- Combine connectivity with the tree edge count
- The commodity is a proof device, not electricity

Draw a disconnected cycle as a counterexample to relying on edge count alone. The model sends a fictitious commodity from the source to each energized non-source bus. Its balance makes a source path necessary; its bounds allow travel only on closed edges. Together with the tree edge count, this certifies a connected radial energized graph in this formulation. There are at most five non-source energized buses, which explains the bound five in the teaching model. This commodity is separate from the electrical power flow P. Ask learners why the source must supply the commodity even though only non-source equations are displayed. The distinction prepares them to read a MILP implementation without confusing graph constraints and physics.

Chen et al. · restoration optimization (2018) · Wang et al. · radiality constraints (2020)

Next: Compare objectives on the same feasible network

Compare objectives on the same feasible network

> **Hold the fault, capacities and constraints fixed.**

Property	Total-service baseline	Priority policy
Same fault and capacities	✓	✓
Same feasible network	✓	✓
Objective	Total kW	Weighted kW
Exact method	Enumeration	MILP + enumeration

- Unweighted baseline: maximize total served kW
- Priority policy: maximize weighted served demand
- Both: identical physical and logical constraints
- Report exact solutions and independent agreement

The classroom baseline is deliberately stronger than an arbitrary greedy order. It exactly maximizes total service over the same feasible graph used by the priority-weighted problem. The baseline is solved by exhaustive enumeration; the weighted problem uses HiGHS MILP and an independent enumeration check. Different algorithms are acceptable here because both solve the same declared finite feasible sets exactly. The comparison isolates the objective policy rather than a solver-quality effect. Report both total and critical demand served. Ask what would make the comparison unfair: granting one method a different tie, relaxing its fault exclusion, changing its loads or silently assigning extra numerical search budget.

> **Discussion**
>
> If critical service improves after changing weights, what caused that change?

Expected observation: The documented objective policy, unless another controlled experiment shows otherwise.

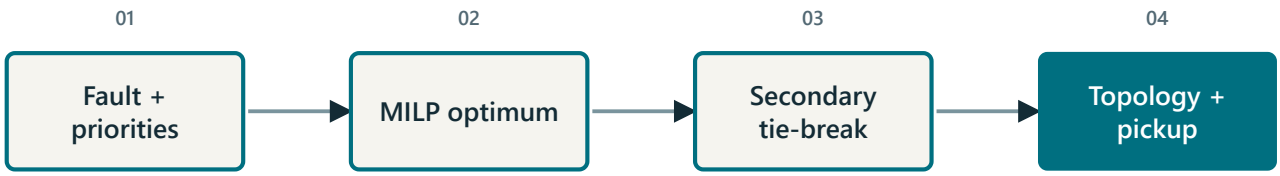
Chen et al. · restoration optimization (2018) · Wang et al. · radiality constraints (2020) · Executed tutorial · Classroom Labs

Next: The planner returns a topology plus a pickup decision

SLIDE 77 · R07

The planner returns a topology plus a pickup decision

> **Use the method that fits the decision type.**



Completed teaching diagram.

- Solve the weighted binary restoration problem
- Preserve the best weighted service in a second solve
- Minimize energized edges, then apply a small tie-break
- Return decisions and solver evidence

Explain why a mixed-integer model is a natural method for on/off topology and load-pickup choices. The first solve maximizes weighted service. A secondary solve preserves that primary optimum, then minimizes energized branch count with a small edge-index tie-break. That secondary quantity is not the number of switching operations relative to an initial topology. If a real restoration policy needs operation counts or crew timing, those variables must be modeled explicitly. MPC could extend this into a rolling multi-period mixed-integer formulation, while a learned policy could propose candidates. Neither extension is executed in this classroom cell. The tutorial keeps the exact standard method and uses orchestration to manage its inputs and evidence.

Chen et al. · restoration optimization (2018) · Wang et al. · radiality constraints (2020) · Executed tutorial · Classroom Labs

Next: L6: a capacity sweep is one reproducible experiment

L6: a capacity sweep is one reproducible experiment

> **Recompute the plan at every tested rating.**

Exact excerpt: Classroom_Labs.ipynb, L6, lines 1–5. Run the complete cell in context.

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
timings['L6 fresh MILP sweep + independent enumeration'] = time.perf_counter() - started
display(restoration['table'].drop(columns='pickup'))
```

- Declare the capacity levels
- Solve weighted and unweighted service problems
- Check the MILP using independent enumeration
- Display total and critical service

Read the five displayed lines as an experiment definition. The capacity tuple identifies five distinct test conditions. The helper solves each one and returns a table containing the objective-policy comparison and independent agreement checks. The measured operation includes the sweep and enumeration, so it is not a single MILP-solve latency. Reusing a previous solution after changing capacity would miss the very effect being studied. Students should save their prediction before running and then inspect the pickup and closed-edge details if a service value is surprising. The final assertion in the full cell checks agreement with exhaustive enumeration for all sampled conditions, giving a direct small-system correctness check.

Executed tutorial · Classroom Labs · Chen et al. · restoration optimization (2018)

Next: Lab: tighten the tie and follow the source path

Lab: tighten the tie and follow the source path

> **Explain discrete service changes using topology and capacity.**

Exact excerpt: Classroom_Labs.ipynb, L6, lines 1–3. Run the complete cell in context.

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
```

- Predict service at 40, 60, 80 and 120 kW
- Run L6 and add one tighter capacity
- Compare critical and total service
- Explain one plateau using the selected loads

Give pairs eight minutes. One participant follows the source path on the network diagram while the other checks capacities and pickup variables in the notebook. Begin with the stated capacity points, then add a tighter point to the tuple and rerun the full cell. Avoid assuming that a twenty-percent capacity reduction causes a twenty-percent service reduction: indivisible pickup and alternative paths create thresholds and flat regions. Learners should compare objective values and feasibility when equivalent topologies tie rather than insisting on identical edge identifiers. If a result appears counterintuitive, trace each served load to source zero and verify every branch capacity. End with a short explanation of one plateau.

> **Experiment / prediction**
>
> Will total service vary smoothly as tie capacity changes?

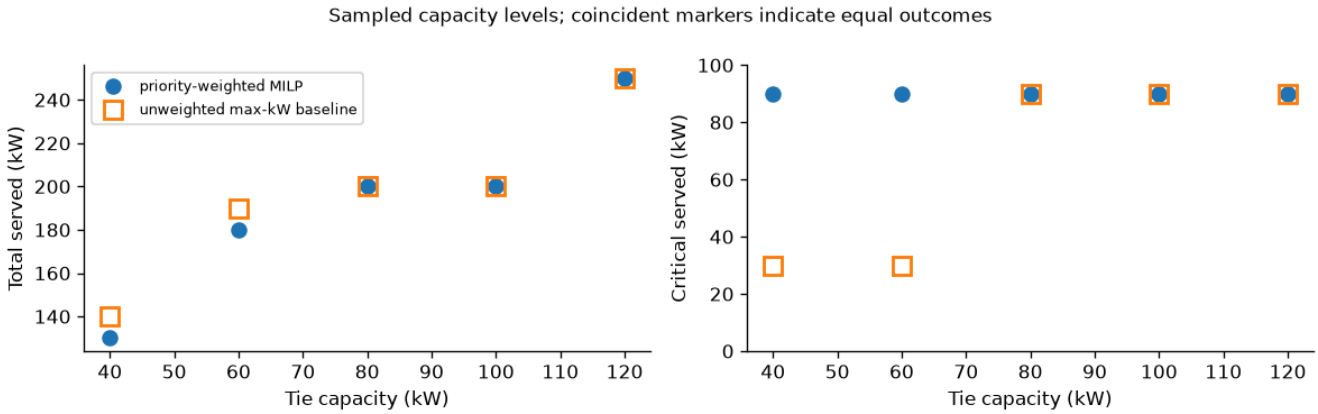
Expected observation: No. Binary load pickup creates thresholds and plateaus.

Executed tutorial · Classroom Labs · Chen et al. · restoration optimization (2018)

Next: The service curve reveals the policy tradeoff

The service curve reveals the policy tradeoff

> **Read critical service and total service together.**



Executed L6 sweep. Markers denote sampled capacities; coincident outcomes overlap. Critical priorities are teaching labels.

- Each marker is a solved capacity level
- Coincident markers mean equal outcomes
- Priority can reduce total kW while preserving critical kW
- Flat regions reflect discrete choices

The plot is generated from the executed classroom sweep. Its markers are discrete solved conditions, not samples of a validated continuous response curve. Weighted and unweighted objectives may select different load combinations, so one policy can restore fewer total kilowatts while restoring more critical kilowatts. Coincident markers show equal outcomes at those conditions. Use the graph to connect the optimization equation with a visible operating consequence. Ask learners to identify one capacity at which reporting total service alone would hide the policy difference. Then ask which field in the operator request justifies the chosen priority. The graph demonstrates the consequences of documented priorities, not an inherent advantage of using an LLM.

Executed tutorial · Classroom Labs · Chen et al. · restoration optimization (2018)

Next: Case A: 40 kW capacity makes priorities visible

Case A: 40 kW capacity makes priorities visible

> **The best total-service plan need not restore the most critical demand.**

40 kW tie case	Total served	Critical served
Unweighted baseline	140 kW	30 kW
Priority-weighted policy	130 kW	90 kW

- Weighted policy: 130 kW total
- Weighted policy: 90 kW critical
- Unweighted baseline: 140 kW total
- Unweighted baseline: 30 kW critical

At the 40 kW swept tie capacity, the priority-weighted solution serves 130 kW in total, including all 90 kW labeled critical in this teaching graph. The unweighted exact baseline serves 140 kW total but only 30 kW critical. These values come from the same feasible network under different objective weights. The ten-kilowatt total-service difference is the price of this particular policy choice in this scenario. It should be shown openly rather than hidden in a weighted score. Ask participants which result matches an instruction to prioritize critical loads, and what additional policy clarification would be necessary if priorities were strict rather than finite weights.

> **Discussion**
>
> Which plan is better if the operator's instruction changes?

Executed tutorial · Classroom Labs · Chen et al. · restoration optimization (2018)

Next: Case B: more capacity can leave service unchanged

Case B: more capacity can leave service unchanged

> **The next available load may need more capacity than the increment supplies.**

- At 80 kW: weighted service is 200 kW
- At 100 kW: weighted service remains 200 kW
- Both serve 90 kW critical demand
- Inspect discrete pickup and the binding path

The weighted solution serves 200 kW total and 90 kW critical at both the 80 and 100 kW swept capacities in the default graph. This plateau makes a useful second case because it prevents a simplistic linear interpretation of capacity improvements. A new indivisible load may not fit the remaining capacity, and another path or constraint may still bind. Have learners inspect the chosen pickup set and closed edges before proposing a causal explanation. The graph alone shows the plateau but does not identify every active constraint. Ask whether splitting a load into controllable blocks would change the problem. It would change the feasible decision set and require a new declared model.

> **Discussion**
>
> Why can a 20 kW rating increase restore no additional load?

Expected observation: The next discrete pickup may remain infeasible, or another constraint still binds.

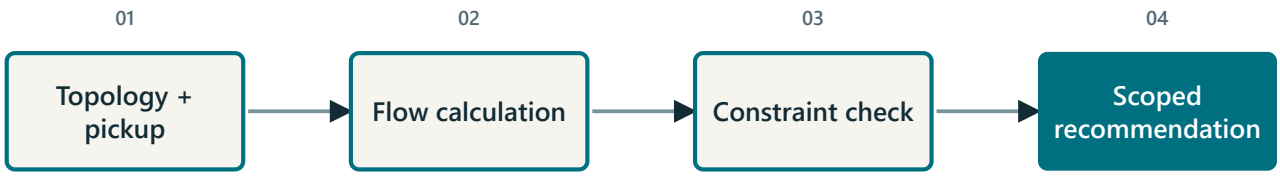
Executed tutorial · Classroom Labs · Chen et al. · restoration optimization (2018)

Next: Simulation and verification check different properties

SLIDE 83 · R13

Simulation and verification check different properties

> **A candidate topology needs recomputation and an independent contract check.**



Completed teaching diagram.

- Simulation: compute flows for the chosen topology
- Verification: capacity, fault exclusion and source connectivity
- Independent check: enumeration agrees with the MILP
- Scope: no AC voltages or switching chronology

Separate the outputs of simulation and verification on the same topology. Simulation calculates the modeled consequences of the candidate. Verification decides whether those consequences and the discrete decisions satisfy the contract. Independent enumeration checks the optimizer's result on this small finite fixture; it does not add AC constraints that the model omitted. The six-node classroom problem includes active-power transport, connectivity and radiality but no voltage, reactive power, relay coordination or time-ordered switching. A real operator must not mistake that scope for a complete switching plan. Ask learners which omitted property would matter before energizing equipment and where its result would enter the workflow.

Executed tutorial · Classroom Labs · Chen et al. · restoration optimization (2018) · Wang et al. · radiality constraints (2020)

Next: L7: changing the rating invalidates old evidence

L7: changing the rating invalidates old evidence

> **A correct old solution is not a checked solution for a new request.**

Exact excerpt: Classroom_Labs.ipynb, L7, lines 2–9. Run the complete cell in context.

```
rest_request = {'task': 'restoration', 'capacity_kw': CAPACITIES_KW[0],
                'critical_nodes': [2, 5], 'model': 'six_node_active_transport', 'revision': 1}
rest_candidate = {'pickup': rest_record['milp']['pickup'], 'closed_edges': rest_record['milp']
['closed_edges']}
rest_evidence = lab.bind_evidence(rest_request, rest_candidate,
    {'objective': rest_record['milp']['objective'], 'enumeration_agrees': rest_record['agreement']})
new_rest_request = {**rest_request, 'capacity_kw': 20., 'revision': 2}
print('Original contract:', lab.bound_to(rest_request, rest_candidate, rest_evidence))
print('Changed rating with old evidence:', lab.bound_to(new_rest_request, rest_candidate, rest_evidence))
```

- Bind the original plan to its request
- Change capacity and revision
- Test the old evidence against the new request
- HOLD and request a fresh constrained solve

Run the full L7 cell after L6. It takes the first weighted restoration result, binds the request and candidate to its evidence, and then changes the capacity to 20 kW with a new revision. The original consistency test succeeds; the changed request with old evidence fails. This is a controlled demonstration of stale-evidence rejection. The hash mechanism does not prove that a simulator was correct or that input data were authentic. It establishes that the evidence record belongs to a different declared request. Ask participants to describe the appropriate operator-visible response in plain language: the earlier recommendation must be held while a new constrained solve is obtained.

> **Experiment / prediction**
>
> Can we keep the old green status after lowering the rating?

Expected observation: No. The changed contract requires new evidence.

Executed tutorial · Classroom Labs

Next: Inspect all five stages in a recorded feeder run

SLIDE 85 · R15 · L10

Inspect all five stages in a recorded feeder run

> **The live MILP lab and the recorded feeder catalog are separate experiments.**

Exact excerpt: Classroom_Labs.ipynb, L10, lines 16–23. Run the complete cell in context.

```
from IPython.display import HTML
for case, backend in [('voltage', 'lindistflow3'), ('restoration', 'lindistflow3'), ('cyber_stealth',
'dc')]:
    full_trace = lab.archived_stage_table(lab.archived_stage_replies(case, backend))
    assert full_trace.stage.nunique() == 5
    display(HTML('<details open><summary>ARCHIVED ACTUAL: ' + case +
        ' — all five stages, every recorded event</summary>' +
        full_trace.drop(columns='role_group').to_html(index=False, escape=True) + '</details>'))
print('Selected returned fields are exact archive values; complete records:
results/proposal_complete_workflow.json')
```

- Awareness: fault isolation and equipment permissions
- Planning: bounded catalog candidates
- Simulation and verification: power flow and thermal rejection
- LLM optimizer: permitted shedding refinement

Make the change of experiment explicit before presenting language responses. L6 and L7 are fresh calculations on a six-node graph. L10 also exposes an archived five-stage feeder workflow using a small synthetic bypass-and-shedding catalog. That archive is not the same MILP experiment. Its awareness record includes a 100 A short bypass, a forbidden long bypass, artificial critical-load labels and no PV dispatch. The planner generates bounded initial catalog candidates. Simulation records physical metrics, and verification rejects the initial high-pickup candidate for thermal failure. The optimizer may request a permitted catalog refinement. Every changed candidate returns to simulation and verification before ranking and operator handoff.

Recorded experiment · five-stage workflow · Executed tutorial · Classroom Labs

Next: A recorded Llama ranking needs numerical scrutiny

SLIDE 86 · R16

A recorded Llama ranking needs numerical scrutiny

> **Explanations are reviewable claims, not substitutes for sorting verified scores.**

- Planning: GENERATE_CANDIDATES · 0.874 s
- Refinement: SHED_075 · 1.259 s
- Later ranking lists E02 before E03
- Recorded costs: −5509.13524 versus −6109.97130

The archived restoration planner asked for both initial simulations within the contract. After a thermal failure, the optimizer selected SHED_075, meaning disconnection of 75 percent of an ordered set of noncritical load objects, not 75 percent of their kilowatts. The later public ranking says lower scores are better yet lists E02 ahead of the lower-cost E03. That mismatch is a concrete reason to keep deterministic verified-score ranking separate from prose. The final numerical selection retains the lower valid cost. Ask learners to explain why negative restoration costs reverse a casual intuition about larger numbers. The recorded call durations are examples from that run, not classroom print times or universal latency benchmarks.

> **Discussion**
>
> Which is lower: −5509 or −6110?

Expected observation: −6110. The verified numeric objective should control the ranking.

Recorded experiment · five-stage workflow

Next: The feeder comparison ties on restored service

The feeder comparison ties on restored service

> **Traceability is demonstrated; extra restored load from Llama is not.**

Archived MV feeder route	Total service	Critical service	Recorded time
Fixed supervisor	1843.614 kW	474.040 kW	0.306 s
Llama supervisor	1843.614 kW	474.040 kW	5.232 s

- Fixed and Llama routes: 1843.614 kW total served
- Both: 474.040 kW critical served
- Fixed route: 0.306 s; Llama route: 5.232 s
- Same selected shedding operation and final model checks

These are recorded end-to-end times for the MV feeder-catalog experiment. Both routes use the same candidate budget and select the same SHED_075 operation, producing the same final service objective. The Llama route makes four language calls and takes about 5.232 seconds, while the fixed route takes about 0.306 seconds. This is one matched scenario, not a statistical latency benchmark or a field deployment. Its value is that the intermediate actions, failed thermal check, public language claims and final evidence are inspectable. Do not use the result to claim additional restored load from agentic AI. A stronger research study would test varied faults, unseen topology conditions and equal computational budgets.

> **Discussion**
>
> Which benefit is supported here: more service or a reviewable workflow?

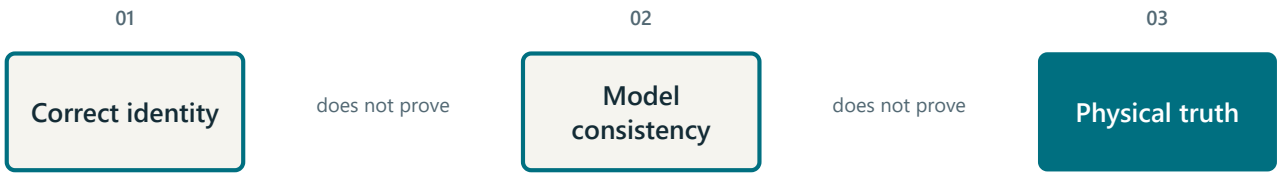
Recorded experiment · five-stage workflow

Next: A matching record can still contain wrong measurements

SLIDE 88 · R18

A matching record can still contain wrong measurements

> **Identity, model consistency and physical truth are different properties.**



Completed teaching diagram.

- Correct identifiers bind a record to a request
- Consistency checks test agreement with assumptions
- Neither property guarantees truthful sensor values
- Next: a corrupted estimate that looks consistent

Conclude restoration with the operator's perspective. The plan packet should show isolated faults, permitted operations, selected pickup, modeled constraints, fresh evidence and any missing checks. It must not present the synthetic snapshot as a verified switching sequence. The stale-evidence exercise solved one problem: using a result for the wrong declared request. It did not solve the deeper problem of untrustworthy measurements inside a correctly identified record. Use this distinction to transition to cyber response. Ask learners whether a well-formed, freshly computed and internally consistent report could still describe the wrong physical state. The next controlled four-bus experiment demonstrates exactly that failure boundary.

> **Discussion**
>
> Could fresh, correctly identified evidence still be physically misleading?

Expected observation: Yes. Its measurements or model assumptions may be wrong.

Executed tutorial · Classroom Labs · Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: A controller acts on an estimate, not on hidden truth

Cyber response

SLIDE 89 · C01

A controller acts on an estimate, not on hidden truth

> **Start with the information available to the operator.**

Operations discussed: Physical state, Measurements, State estimate, Control proposal.

- Four-bus DC teaching system
- Three unknown non-reference voltage angles
- Eight flow, injection and angle measurements
- Two corruption patterns; one controlled experiment

Introduce the data path from physical state to measurements to estimator to controller. The four-bus fixture is separate from the Greensboro feeder and is designed to expose a detector limitation clearly. Its unknown state contains three non-reference voltage angles, while eight measurements provide redundancy. The experimenter knows the simulated true state for evaluation, but the agent's operating gate does not receive that truth. Two numerical corruption patterns are compared: one isolated sensor bias and a coordinated pattern consistent with the same linear model. The objective is to understand information limits and safe handling of uncertainty. Ask which quantities an operator can observe directly and which are inferred.

> **Discussion**
>
> Does a precise-looking state estimate guarantee an accurate state?

Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: WLS combines measurements using their uncertainty

WLS combines measurements using their uncertainty

> **A model and covariance turn measurements into an estimate.**

$$z = Hx + \epsilon, \quad \epsilon \sim \mathcal{N}(0, R), \qquad \hat{x} = \arg\min_{x} (z - Hx)^{\top} R^{-1} (z - Hx)$$

$x, \hat{x}$	True and estimated non-reference angles, rad; dimension 3	z, H	Eight measurements and the linear measurement matrix
ϵ, R	Measurement noise and its covariance		

- H maps angles to expected measurements
- R weights sensors in their own units
- The estimate minimizes weighted mismatch
- Observability requires sufficient independent information

Read the measurement relation before the minimization. The state vector x contains three angles in radians. The vector z contains eight sensor readings with potentially different physical units. H maps angles to expected measurements, and covariance R accounts for their assumed uncertainty. A measurement with lower variance receives greater influence after proper weighting. Full column rank is required to identify all state directions in this linear fixture. In numerical code, use a stable weighted least-squares solve rather than treating an explicit matrix inverse as the preferred implementation. Ask learners why simply averaging all measurement errors would be inappropriate when flow, injection and angle channels have different scales and uncertainties.

Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: A residual tests consistency with assumptions

A residual tests consistency with assumptions

> **No alarm means no detected inconsistency under the stated model.**

$$r = z - H\hat{x}, \qquad J = r^{\top} R^{-1} r, \qquad J > \chi^2_{5,0.99} = 15.0863 \;\Rightarrow\; \text{alarm}$$

r	Measurement residual	J	Dimensionless residual energy
$\chi^2_{5,0.99}$	99th percentile with 5 residual degrees of freedom		

- Residual: unexplained measurement mismatch
- Weighted energy J is dimensionless
- Eight measurements minus three states gives five degrees
- 99% threshold: 15.0863

Define the residual as what the estimated state cannot explain. Its covariance-weighted energy J is compared with a chi-squared threshold under the assumed linear model and Gaussian noise. Here the residual has five degrees of freedom because there are eight measurements and three observable state variables. The 0.99 quantile is approximately 15.0863. The nominal one-percent false-alarm level depends on the model, noise and rank assumptions; it is not a measured field false-alarm rate. An exceedance indicates inconsistency, not malicious intent or the identity of a compromised sensor. Ask the audience to finish the sentence: no residual alarm means no inconsistency was detected by this test, not that the state is true.

> **Discussion**
>
> Does an alarm establish that the cause was malicious?

Expected observation: No. Model error, noise assumptions and ordinary sensor faults can also cause inconsistency.

Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: Case A: bias one measurement

Case A: bias one measurement

> **An isolated error often leaves a residual signature.**

- Change one sensor channel
- Leave the other channels unchanged
- Re-estimate the state
- Inspect the weighted mismatch

Use the isolated corruption as a familiar reference before the coordinated case. Adding bias to one measurement can create a disagreement that the remaining redundant channels cannot explain. As the bias grows, residual energy tends to increase in this chosen fixture. Avoid turning that observation into a claim that every isolated attack is detectable or that any alarm proves an attack. Detection depends on the measurement arrangement, uncertainty, bias direction and magnitude. The later lab computes the result rather than assuming it. Ask learners to predict which curve should rise first: the residual statistic or the state-estimation error. Both can change, but they measure different aspects of the outcome.

> **Discussion**
>
> Which other measurements could contradict the biased sensor?

Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: Case B: shift measurements along a modeled direction

Case B: shift measurements along a modeled direction

> **A structured perturbation can look like a different physical state.**

$$a = Hc, \qquad z' = z + a = z + Hc$$

c	Chosen state displacement, rad	a	Coordinated measurement perturbation
z'	Perturbed measurement vector		

- Choose a state displacement c
- Map it into measurement space using H
- Add that pattern to the measurements
- Use the unchanged estimator model

Introduce c as a hypothetical state displacement and a as a measurement-space perturbation. The construction a=Hc is a mathematical teaching case: the added measurement pattern is exactly one that the assumed linear model can explain. It requires knowledge of the relevant model and the ability to alter the required channels. Those conditions are important limits, not details to hide. The purpose is to test the detector's information boundary in a controlled fixture, rather than claim every real system permits this pattern. Ask learners what state estimate would explain z plus Hc if the original estimate explained z. Their prediction leads directly to the cancellation on the next slide.

> **Discussion**
>
> What estimated state would explain the added Hc term?

Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: Cancel Hc and see the detector limitation

SLIDE 94 · C06

Cancel Hc and see the detector limitation

> **The estimate can move while the residual stays unchanged.**

$$r' = z' - H\hat{x}' = z + Hc - H(\hat{x} + c) = z - H\hat{x} = r$$

$\hat{x}' = \hat{x} + c$ Shifted estimate under the same full-rank linear WLS model

$r' = r$ Exactly unchanged residual for this construction

- With the same full-rank WLS model: x̂′ = x̂ + c
- Substitute into the residual
- The added Hc terms cancel
- No residual alarm can coexist with a wrong estimate

Keep the measurement relation visible while revealing the cancellation in steps. For the same full-rank linear WLS problem, adding Hc to the observations shifts the estimate by c. Substituting that shifted estimate into the residual removes the added terms exactly. The residual is unchanged even though the estimated state has moved. Give the audience a few seconds to perform the cancellation before showing the result. This is a detector limitation under specified assumptions, not a universal claim about all estimators or attacks. A language model explaining the same residual more fluently does not create missing information. The next lab contrasts the unchanged detector statistic with an offline state-error metric.

> **Discussion**
>
> What does the detector fail to observe here?

Expected observation: A state displacement that remains consistent with the measurement model.

Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: Awareness should preserve provenance and uncertainty

Awareness should preserve provenance and uncertainty

> **Do not turn a weak observation into a strong assurance.**

- Record sensor provenance and trust assumptions
- Check estimator rank and data validity
- Report residual alarm status accurately
- Keep unknown authenticity explicit

Return from algebra to the awareness stage. Its output should include measurement sources, the assumed covariance, estimator observability and the residual test result. It should also identify which trust assertions are external assumptions. A matching record identifier does not authenticate its sensor values, and a low residual does not demonstrate their physical truth. The correct public explanation may be that no inconsistency was detected by this estimator while coordinated corruption remains possible under the model. Such wording is useful to an operator because it preserves the uncertainty needed for the next decision. Ask learners which additional evidence would actually change the information available, rather than merely restating the same readings.

> **Discussion**
>
> Would another LLM reading the same residual add independent sensor evidence?

Expected observation: No. Independent information requires a justified additional source.

Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: L8: run two corruption families with one shared estimator

L8: run two corruption families with one shared estimator

> **Sweep the perturbation size and retain comparable conditions.**

Exact excerpt: Classroom_Labs.ipynb, L8, lines 1–5. Run the complete cell in context.

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
timings['L8 fresh WLS attack sweep'] = time.perf_counter() - started
display(cyber['table'])
```

- Use the same scale grid
- Compute isolated and coordinated cases
- Record residual and offline state error
- Time the whole WLS sweep

Walk through the first five lines of L8. ATTACK_SCALES defines the tested magnitudes, and the helper recomputes the WLS outcomes for both perturbation families. The same fixture, covariance and threshold keep the comparison interpretable. The table includes a detector-visible statistic and an offline error calculated from known synthetic truth. The latter is available to the teacher for scoring, not to the operating gate. The timer encloses the complete helper call and should not be presented as one sensor-update latency. Ask participants to sketch the expected residual curves before running. The experiment becomes more instructive when learners commit to a prediction rather than merely observing a plot.

Executed tutorial · Classroom Labs · Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: Lab: separate detector evidence from evaluation truth

Lab: separate detector evidence from evaluation truth

> **Find a case with growing error and no residual alarm.**

Exact excerpt: Classroom_Labs.ipynb, L8, lines 1–3. Run the complete cell in context.

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
```

- Predict both residual curves
- Run L8
- Mark the 15.0863 threshold
- Explain which plot an operator can actually use

Allow seven minutes for the two-family sweep. Learners should identify at least one model-consistent corruption level at which the residual remains below threshold while the offline state error increases. Have them label the residual plot as detector-visible and the state-error plot as evaluation-only. The truth metric must not quietly enter the decision logic. Encourage learners to add a scale value and observe whether the algebraic prediction persists in the numerical calculation. Ask what assumptions would need to change to break the exact cancellation. Answers include restricted channel access, a different measurement model or additional independent information. Finish with one careful sentence explaining what no alarm does and does not mean.

> **Experiment / prediction**
>
> Which curve can be computed without knowing the true state?

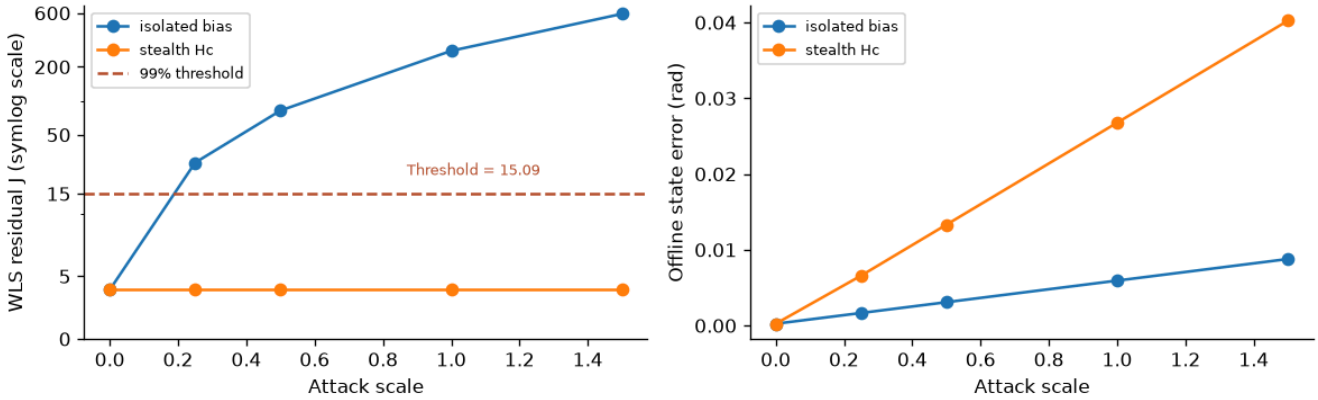
Expected observation: The WLS residual curve; state error requires the synthetic truth.

Executed tutorial · Classroom Labs · Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: One curve is flat while the state error grows

One curve is flat while the state error grows

> **Model consistency and state accuracy answer different questions.**



Executed L8 sweep. Left: detector-visible residual. Right: offline state error using synthetic truth.

- Coordinated residual J stays near 3.92022
- Isolated scale-1 residual J is about 277.497
- Threshold is 15.0863
- The right-hand error curve uses offline truth

Use the plotted default outputs to connect the cancellation to a visible result. The coordinated case preserves a residual energy of approximately 3.92022 across the tested scales, below the threshold of 15.0863, while its offline state error grows. The isolated scale-one case has residual energy around 277.497 and exceeds the threshold. The residual axis uses a symmetric-log presentation to retain detail near the threshold and at larger values; read the tick labels carefully. These are outcomes of the controlled fixture, not estimates of field detection rates. Ask participants to explain why adding another prose interpretation of the flat curve cannot by itself reveal the hidden displacement.

Executed tutorial · Classroom Labs · Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: Plan recovery from a justified information source

Plan recovery from a justified information source

> **Protected measurements change the state information used by the planner.**

$$\min_{0\le u\le 0.5} \frac{1}{2}u^2 \quad \text{s.t.} \quad |F\hat{x}_{\mathcal{T}} + b_u u| \le \overline{F} - 0.01$$

u	Added node-2 injection, balanced at source, p.u.	$\hat{x}_{\mathcal{T}}$	State estimated from assumed protected readings
F, b_u	Angle-to-flow map and control sensitivity	$\overline{F}$	Branch limits (0.35, 0.18, 0.35, 0.35) p.u.

- Estimate from three assumed protected angle readings
- Choose the smallest corrective injection
- Keep predicted branch flows inside tightened limits
- HOLD if protected information is unavailable

Introduce the trust assumption before the optimization. The fixture provides three additional angle readings explicitly assumed protected; neither the estimator nor the LLM establishes that protection. From their estimate, a scalar quadratic program chooses added injection at node two, balanced at the source. The objective minimizes corrective effort, and the branch-flow constraints include a 0.01 p.u. planning margin. This is a standard constrained model-based response, not an MPC rollout or trained SAC action in this case. The hard bounds and trust basis remain visible in the returned record. Ask learners why changing only the prose label from suspicious to trusted would not justify this calculation.

Executed tutorial · Classroom Labs · Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: Three independent readings remove redundancy

SLIDE 100 · C12

Three independent readings remove redundancy

> **A full-rank estimate can still lack a residual consistency test.**

$$\operatorname{rank}(H_{\mathcal{T}}) = 3, \qquad m_{\mathcal{T}} - n = 3 - 3 = 0$$

$H_{\mathcal{T}}$	Measurement matrix for the protected subset	$m_{\mathcal{T}}, n$	Protected measurement count and state dimension

- Three protected angle readings
- Three unknown state variables
- Rank 3 permits estimation
- Residual degrees of freedom: zero

Pause on a subtle consequence of the protected-channel fixture. Three independent angle readings can identify the three unknown state variables, but they leave no residual degrees of freedom. The estimate exists because of rank, while the trust in those readings remains an external assumption. Do not describe zero residual as confirmation that the protected readings are correct. This is an excellent example of why observability, redundancy and authenticity should be taught separately. Ask the audience how they would strengthen the evidence in a real system: authenticated channels, independent measurements, validated sensor health and appropriate threat assumptions are distinct contributions. In this tutorial, unavailable protected information leads to HOLD rather than an invented trusted state.

> **Discussion**
>
> Does full rank imply enough redundancy to test consistency?

Expected observation: No. This exactly determined protected subset has zero residual degrees of freedom.

Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: L9: compare trusted evidence with its absence

L9: compare trusted evidence with its absence

> **The correct unavailable-data result is an explicit HOLD.**

Exact excerpt: Classroom_Labs.ipynb, L9, lines 1–9. Run the complete cell in context.

```
without_trust = lab.trusted_response(cyber, protected_available=False)
with_trust = lab.trusted_response(cyber, protected_available=True)
display(pd.DataFrame([
    {'condition': 'No trusted replacement channels', 'status': without_trust['status'],
     'u_pu': None, 'offline_actual_violations': None},
    {'condition': 'Three trusted channels assumed', 'status': with_trust['status'],
     'u_pu': with_trust.get('flexible_injection_pu'),
     'offline_actual_violations': with_trust.get('actual_violation_count')}]))
print(with_trust['assumption'])
```

- Call the same response function twice
- Protected unavailable → HOLD
- Protected available → constrained injection proposal
- Inspect the stated trust assumption

Read both helper calls before showing the table. The first removes the protected-channel assumption and must return HOLD. The second enables the stated protected readings, solves the constrained recovery problem and reports an advisory candidate. This comparison teaches an information dependency, not a difference between a weak and a strong language model. The assertions in the full cell verify the expected hold behavior and the absence of offline actual violations in this fixed scenario. The offline check uses synthetic truth solely to evaluate the experiment. Ask learners what a control-room display should retain when it holds: the missing evidence, the reason for holding and a clear path for obtaining justified additional information.

Executed tutorial · Classroom Labs · Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: Lab: connect the new estimate to a physical consequence

SLIDE 102 · C14

Lab: connect the new estimate to a physical consequence

> **Explain the chain from information to command to branch flow.**

- Run L9
- Check the 0.18 p.u. limit on branch 1–2
- Inspect every other branch
- Describe why protected information changes the action

Give learners five minutes to run L9 and compare the two information conditions. The useful causal chain is protected evidence changes the estimate, the estimate changes the constrained injection, and that injection changes the modeled physical flows. A label from an LLM is not the source of the new information. Have participants inspect all branches rather than stopping after the highlighted branch falls below its limit. The 0.01 p.u. planning margin is an engineering choice and is not calibrated as a probabilistic guarantee. If someone changes the fixture, the displayed reference numbers may no longer apply; they should report their newly calculated values and the assumptions behind them.

> **Experiment / prediction**
>
> What causes the improved physical outcome here?

Expected observation: The protected-state estimate and the constrained numerical control, under the stated assumptions.

Executed tutorial · Classroom Labs · Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: The control consequence makes the failure visible

The control consequence makes the failure visible

> **A plausible corrupted estimate can leave a real overload untreated.**

Information basis	Injection u	Actual flow 1–2
Stealth-corrupted	0 p.u.	0.24000 p.u.
Assumed protected	0.12169 p.u.	0.17046 p.u.

- Corrupted-state control: u = 0
- Its actual branch 1–2 flow: 0.24000 p.u.
- Protected-state control: u = 0.12169 p.u.
- Its actual branch 1–2 flow: 0.17046 p.u.

Compare the two commands under the same known synthetic truth. Using the stealth-corrupted estimate leaves the corrective injection at zero and the highlighted actual branch flow at 0.24 p.u., above the 0.18 limit. Using the protected estimate gives approximately 0.12169 p.u. injection and 0.17046 p.u. actual flow on that branch. The slight difference from the planned tightened bound reflects the fixture's measurement noise. All other branch limits must also be checked. These actual-flow values are evaluation outcomes, not measurements secretly given to the operating gate. The result demonstrates the importance of the information basis and constrained controller; it does not establish a universal cyber-defense guarantee.

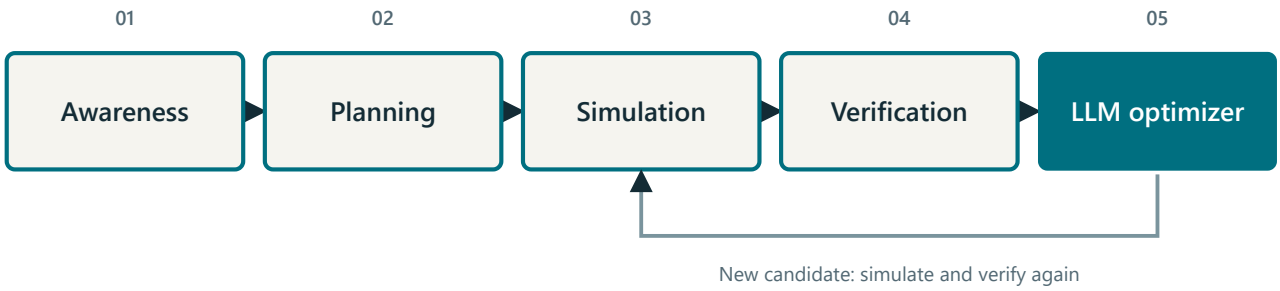
> **Discussion**
>
> Why is the actual-flow column labeled evaluation-only?

Executed tutorial · Classroom Labs · Schweppe & Wildes · state estimation (1970) · Liu, Ning & Reiter · false-data injection (2009)

Next: Preserve uncertainty through all five stages

Preserve uncertainty through all five stages

> **The architecture should carry trust assumptions to the operator.**



Completed teaching diagram.

- Awareness: provenance, rank and residual evidence
- Planning: WLS estimate and constrained QP action
- Simulation and verification: predicted flows and binding
- LLM optimizer: permitted refinement, public rationale or escalation

Animate the same five-stage structure used in the previous cases, but emphasize the changed numerical methods. Awareness records provenance and estimator diagnostics. Planning uses a WLS estimate and a scalar constrained QP. Simulation computes DC flows for the candidate. Verification checks trust-basis requirements, rank, action bounds, flow limits and evidence identity. The LLM optimizer may request a permitted numerical refinement and provide a public explanation, but cannot manufacture protected measurements. A new estimate or command triggers another numerical check. The operator packet retains the assumed trust basis and remaining limitations. This general architecture accommodates different domain solvers without requiring MPC or SAC to be used in every decision type.

Executed tutorial · Classroom Labs · Recorded experiment · five-stage workflow · Schweppe & Wildes · state estimation (1970)

Next: Recorded Llama refinement does not improve the final control

Recorded Llama refinement does not improve the final control

> **A numerical baseline remains the reference for claimed agentic benefits.**

Archived cyber route	Final u	Recorded total
Fixed supervisor	0.12169 p.u.	0.0038 s
Llama supervisor	0.12169 p.u.	5.203 s

- Planning reply: GENERATE_CANDIDATES · 0.829 s
- TRUSTED_QP refinement reply: 1.195 s
- Fixed and Llama routes select u = 0.12169 p.u.
- Recorded totals: fixed 0.0038 s; Llama 5.203 s

The archived cyber-stealth workflow is inspected in L10, separately from the fresh L8 and L9 calculations. Its actual planning response asks for new candidates based on the given tools and objective. A later optimizer reply chooses TRUSTED_QP and requests new simulation and verification. The public response is useful because its proposed operation and stated justification can be checked. However, the fixed and Llama routes finish with the same constrained injection and objective. Their recorded total times differ substantially because the Llama route makes four model calls. These are single-run measurements, not a distributional benchmark. The example demonstrates reviewable orchestration and the need for numerical checks, not improved attack detection or lower control cost from the language layer.

> **Discussion**
>
> Actual planning reply: “Requesting new candidates based on the given tools and objective.”

Expected observation: Inspect the returned candidate and fresh numerical evidence before accepting the explanation.

Recorded experiment · five-stage workflow · Executed tutorial · Classroom Labs

Next: Three cases, three different failure boundaries

Three cases, three different failure boundaries

> **A common workflow manages evidence; each use case still needs its own physics.**

Case	Failure boundary	Evidence response
Voltage	Omitted constraints	Broader-model audit
Restoration	Stale recommendation	Fresh case-bound solve
Cyber	Consistent wrong state	Justified trusted source / HOLD

- Voltage: reduced scope can omit binding limits
- Restoration: changed conditions invalidate an old plan
- Cyber: consistent data can describe the wrong state
- Common response: justified tools, fresh checks and operator review

Give participants a minute to write one sentence for each case. Voltage support showed that a reduced-model PASS may fail a broader physical audit. Restoration showed that priority policy changes service tradeoffs and that a changed capacity invalidates old evidence. Cyber response showed that a model-consistent perturbation can preserve residuals while changing the estimated state. The common framework keeps requests explicit, selects appropriate numerical methods, records their outputs, verifies changed proposals and carries limitations to a human reviewer. It does not create physical information by adding language agents. The final research discussion should ask when learned orchestration improves task success enough to justify its latency, complexity and additional failure modes.

> **Discussion**
>
> Which stage would you strengthen first for each failure boundary?

Executed tutorial · Classroom Labs · Recorded experiment · LLM response selection · Recorded experiment · five-stage workflow

Next: What did the framework actually improve?

Evaluation

SLIDE 107 · E01

What did the framework actually improve?

> **Measure the workflow and the electrical outcome separately.**

- Study completed?
- Constraints passed?
- Claims supported?
- Operator decision improved?

Ask learners to answer this using one case they just ran. A study can finish while its candidate is infeasible. A rejected candidate can demonstrate a working gate while consuming time. A numerical improvement can come from a stronger optimizer rather than the LLM. Put those observations in different columns. The tutorial establishes an executable learning workflow with visible intermediate decisions; it does not yet establish a benefit in a utility control room. Invite the audience to name the smallest extra experiment needed for their preferred claim. A claim about staff workload requires participants and a workload measure. A claim about electrical feasibility requires a stated model, operating limits and test scenarios. Transition: make the comparison fair before interpreting any score.

> **Discussion**
>
> Which claim is supported by a low cost and a fluent explanation?

Expected observation: Only the measured cost is established; feasibility and explanation support require separate evidence.

PowerAgentBench-SS (2026), §§II–III · Executed tutorial: Classroom_Labs, L0–L12

Next: Compare methods on the same task

Compare methods on the same task

> **Give every method the same physical constraints and verification budget.**

$$\hat{p}_{\text{pass}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[g(x_i, u_i) \le 0]$$

N	number of evaluated cases	g	vector of checked constraint violations
$1[\cdot]$	one when every listed constraint passes	i	case index
x_i	evaluated state in case i	u_i	candidate control in case i

- Same case and admissible actions
- Same candidate trial budget
- Same final verifier
- Separate failures from feasible costs

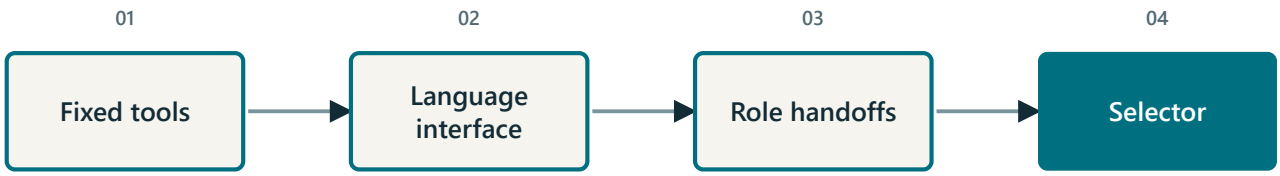
Use the same demand realization, topology, operating limits, action bounds and final electrical check for a paired comparison. A method that sees more cases or gets extra simulations has received additional information or compute. Record that explicitly. The fixed baseline may use the same MPC, SAC policy and verifier as the agentic workflow; this isolates orchestration rather than changing the entire controller. Report the objective among feasible candidates and also report the failure fraction. An average objective that silently removes failed cases can favor a brittle method. In the held-out response pilot, all pool selectors paid for all three LLM calls. The learned selector therefore cannot claim the latency of only the chosen response. Transition: choose the ablation that answers the claim.

PowerAgentBench-SS (2026), §§II–III · Recorded voltage pilot: E1–E8

Next: Ablate one mechanism at a time

Ablate one mechanism at a time

> **A useful ablation keeps the numerical tools fixed.**



Completed teaching diagram.

- Fixed numerical workflow
- Add language interpretation
- Add role-specific handoffs
- Add vote or learned selection

Build the comparison from left to right. First establish a deterministic workflow that solves the engineering problem. Add language interpretation while preserving its tool contract. Then introduce specialist handoffs and measure whether they reduce malformed requests or unsupported reports on varied tasks. Finally compare selection rules under the same candidate pool and fresh-trial budget. This is a recommended experimental design, not a claim that every ablation was run in this package. The implemented pilot directly compares several response selectors and fixed numerical refinement; the real LangGraph exercise separately checks orchestration equivalence against direct Python. Keep those scopes visible. Ask which variant would isolate the value of retrieval or a checkpoint. Transition: a recent benchmark formalizes this separation between tools and scoring.

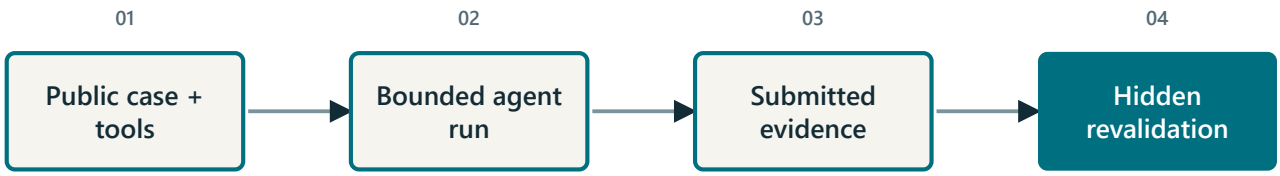
PowerAgentBench-SS (2026), §§II–III · Executed LangGraph teaching lab: G0–G5 · Recorded voltage pilot: E1–E8

Next: Frontier: evidence-backed evaluation

SLIDE 110 · E04

Frontier: evidence-backed evaluation

> **The final report should survive independent recomputation.**



Completed teaching diagram.

- ✓ Public tools and fixed budgets
- ✓ Hidden physical revalidation
- ✓ Evidence-supported submissions
- Scope: DC thermal pilot

PowerAgentBench-SS separates an agent-visible tool environment from hidden evaluation. Its instantiated pilot studies DC thermal contingency analysis, so it is useful here as an evaluation pattern rather than evidence for our voltage controller. For the classroom, draw a boundary around what the agent may observe and place the final scoring calculation outside it. Training feedback and selector features must not contain the held-out answer. A correct statement can still lack supporting evidence in the submitted trace, which is why claim support deserves its own measure. Have a learner point to the tool record supporting one numerical sentence in an operator report. If the record is absent, mark the sentence unsupported even if its value happens to be true. Transition: operating uncertainty is another dimension the workflow must expose.

PowerAgentBench-SS (2026), §§II–III

Next: Frontier: request uncertainty studies

SLIDE 111 · E05

Frontier: request uncertainty studies

> **The orchestrator can request a stronger study when uncertainty matters.**

- ✓ Numerical engines remain explicit
- ✓ Uncertainty-aware study selection
- ✓ Corrective-dispatch tools
- Measurement replay is not forecast validation

CONDUCTOR provides a recent example of language orchestration around numerical grid-analysis engines, including uncertainty-aware analysis. Its reported measurement replay must not be described as a forecast-performance experiment. Relate that distinction to our load-scale exercise: changing a scenario is a sensitivity study, while a probability claim needs a justified distribution and calibration. A useful extension is to provide a small declared scenario set, run the same candidate through every scenario, and summarize the observed margins. That extension is proposed here; it is not an experiment already contained in the saved classroom output. Ask what additional data would justify the chosen scenarios for a real feeder. Transition: uncertainty about the grid and authority to act are separate concerns.

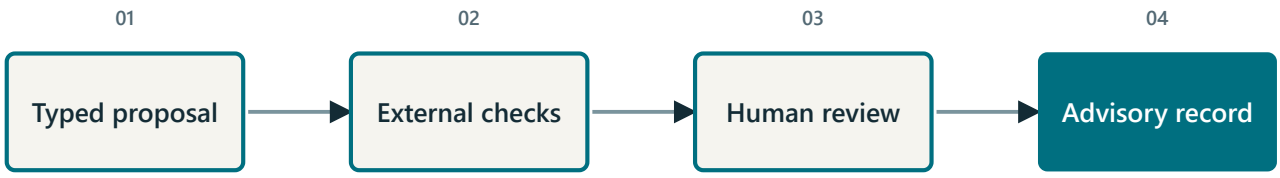
CONDUCTOR (2026), uncertainty-aware orchestration · Executed tutorial: Classroom_Labs, L0–L12

Next: Frontier: enforce authority in software

SLIDE 112 · E06

Frontier: enforce authority in software

> **Approval rules belong in the runtime, not only in the prompt.**



Completed teaching diagram.

- ✓ Whitelisted tools
- ✓ Explicit step budgets
- ✓ Typed numerical facts
- ✓ Review before side effects

The governance-aware TSO digital-twin paper uses an external control layer for tool permissions, budgets and approval. Its reported prototype distinguishes task success from compliance with those enforced rules. Bring that distinction into our notebook: a candidate can satisfy the advisory readback rule without becoming a field command. Conversely, a plausible language answer cannot make an unavailable permission appear. Draw the authorized path through the review gate and show the attempted bypass ending at HOLD. This is an architectural lesson, not a claim of production certification. Ask learners to identify which checks run outside the language model in L12. The answer includes candidate identity, evidence binding and required model scope. Transition: now test what the existing experiments actually support.

Governance-aware TSO digital twin (2026) · Executed tutorial: Classroom_Labs, L0–L12

Next: L11: one optional local Llama call

L11: one optional local Llama call

> **An actual model response proposes a trial; the numerical verifier still decides admissibility.**

Exact excerpt: Classroom_Labs.ipynb, L11, lines 6–8. Run the complete cell in context.

```
client = ProposalLlama(journal, model='llama3.1:8b')
live_reply = client.ask('Optimization Agent', pool['context']['public_context'],
                        list(pool['context']['catalog']))
```

- Default: no new model call
- Enable RUN_LOCAL_LLAMA in L0 for a new call
- The saved response is from original test_01

Run the complete L11 cell after the earlier engineering cells. Its default branch prints that no new model call occurred. The optional path requires RUN_LOCAL_LLAMA=True in L0 and a prepared local Ollama service containing llama3.1:8b. It asks for a choice from the bounded catalog on the original test_01 context, then writes the actual response to a new journal. Do not silently apply that response to a modified classroom case. The quoted public reason comes from the separate 17 October validation and is a saved actual output, not a new live response generated by these slides. It proposes reducing reactive effort and uses cautious language about possible cost improvement. A fresh numerical trial is still required. Transition: inspect where the measured call spent its time.

> **Discussion**
>
> Recorded Llama reason: “has the potential to reduce reactive effort magnitude by ten percent, which may improve the cost.”

Expected observation: The 17 October validation chose RELAX_Q_10_PERCENT. Test it before accepting it.

Ollama chat API · Classroom L11 and saved actual validation

Next: Where did one Llama call spend its time?

SLIDE 114 · E08

Where did one Llama call spend its time?

> **Report loading, prompt processing and generation separately.**

Measured component	Seconds
Load	16.967
Prompt	15.192
Generate	1.916
Other client time	0.038

- Load: 16.967 s
- Prompt: 15.192 s
- Generation: 1.916 s
- Client elapsed: 34.113 s

This is the saved actual Llama 3.1 8B call from the optional classroom validation, not a new call made by the slide. Ollama reports its loading, prompt-processing and generation durations; the client timer includes the complete request. Small differences include server and transport overhead, so do not force the components to equal the client total exactly. The response chose RELAX_Q_10_PERCENT on the original test_01 context and requested a newly checked trial; it did not certify that future trial. Warm loading, prompt length and hardware can change latency substantially. Ask the audience whether this call belongs in a subsecond control loop or an advisory study. A deadline claim would require repeated end-to-end measurements, including failures and fallback. Transition: present this evidence in an operator-facing packet.

Ollama chat API: timing fields · Executed tutorial: Classroom_Labs, L0–L12

Next: Place the assistant beside the control-room tools

Control room

SLIDE 115 · O01

Place the assistant beside the control-room tools

> **The tutorial is an advisory sandbox with an explicit human boundary.**



Conceptual AI-generated illustration, not a utility facility or deployed system.

- Existing operational data interfaces
- Sandboxed numerical studies
- Evidence-backed recommendation
- Authorized operator procedure

Follow the data path from the existing operational applications to a read-only study context, then through numerical candidates and verification to a reviewable recommendation. This is a deployment concept. The notebooks use a synthetic feeder and teaching cases; they do not connect to a live control system. The diagram should make the human review point prominent without suggesting that a dashboard alone establishes useful human oversight. An operator needs the relevant evidence at the right time and must be able to reject or request clarification. Ask which information belongs in a structured field rather than generated prose: equipment identifiers, limits, model scope and candidate identity are strong examples. Transition: open one packet and inspect its information hierarchy.

Governance-aware TSO digital twin (2026) · Executed tutorial: Classroom_Labs, L0–L12

Next: The operator sees scope before readback

SLIDE 116 · O02

The operator sees scope before readback

> **Lead with action, scope, margins and unresolved issues.**

Packet field	Full MV/LV request	Explicit MV teaching request
Verified scope	MV phases only	MV phases only
Scope matches	No	Yes
State	HOLD	READY_FOR_READBACK
Acknowledged	False	True
Equipment dispatch	False	False

- What action and candidate ID?
- Which model and data revision?
- What passed, failed or remains unknown?
- What acknowledgment is required?

Present the action and its identity first, then the supporting evidence. A reviewer should not have to reconstruct the proposal from a long chat transcript. Include the requested and verified model scope, the age of the evidence, numerical constraint margins, rejected alternatives and a concise public rationale tied to tool results. In this tutorial the most consequential scope distinction is the MV-only result versus the broader MV/LV requirement. A polished summary cannot silently expand that scope. Invite learners to remove one field and predict the resulting ambiguity. Removing the identifier obscures which candidate was reviewed; removing the scope can turn a valid limited statement into an unsupported general claim. Transition: acknowledgment changes a specific state, not the physical network.

> **Discussion**
>
> Both packets refer to b67cff9c8eb444c2. What changes the result?

Expected observation: The requested scope must match the verified scope; the recorded readback remains advisory.

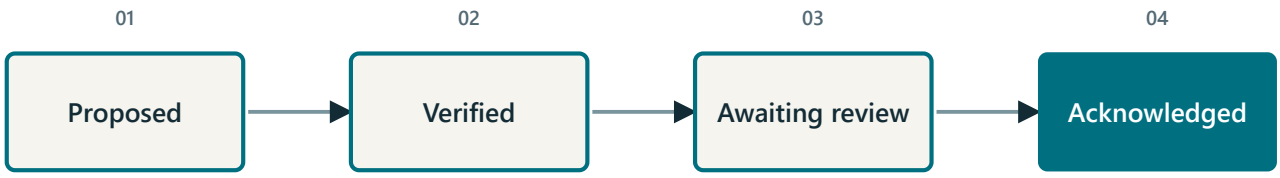
Executed tutorial: Classroom_Labs, L0–L12 · Governance-aware TSO digital twin (2026)

Next: Acknowledgment is a guarded transition

SLIDE 117 · O03

Acknowledgment is a guarded transition

> **Only a matching, eligible packet can be acknowledged.**



Completed teaching diagram.

- Proposed → checked
- Checked → awaiting review
- Matching readback → acknowledged
- Scope or identity failure → HOLD

Animate the review path one state at a time. The conceptual sequence distinguishes calculation, review readiness and human acknowledgment. In the implementation, a full MV/LV request supported only by an MV result is held; an explicitly limited MV teaching request can become eligible for acknowledgment. The candidate identifier binds the readback to a particular packet. Emphasize that these are advisory application states. They neither authenticate a real operator nor dispatch equipment. The earlier LangGraph G5 exercise demonstrated interruption and resumption using a checkpoint, while L12 demonstrates the engineering packet checks. They teach related ideas through separate code paths. Ask the audience to identify which failure should stop the path before any readback is accepted. Transition: inspect the exact L12 statements.

Executed tutorial: Classroom_Labs, L0–L12 · Executed LangGraph teaching lab: G0–G5 · LangGraph: interrupts and resume

Next: Code: bind review to the candidate

Code: bind review to the candidate

> **A wrong candidate ID cannot acknowledge the packet.**

Exact excerpt: Classroom_Labs.ipynb, L12, lines 9–10. Run the complete cell in context.

```
wrong_id = lab.acknowledge(mv_scope, 'wrong-id', 'Student operator')
readback = lab.acknowledge(mv_scope, mv_scope['recommendation_id'], 'Student operator')
```

- First request uses a wrong ID
- Second reads back the packet ID
- The operator label is a classroom placeholder

Read these two notebook lines aloud. The first call deliberately supplies the wrong recommendation identifier. The second uses the identifier carried by the eligible MV teaching packet. Both calls go to ordinary Python logic, so the readback rule is inspectable independently of language-model output. The string Student operator is an instructional placeholder rather than an authentication mechanism. Keep the previously constructed packet visible: it records what was checked and under which scope. This exact excerpt comes from L12 of Classroom_Labs, and learners should run L0–L5 first so the selected-trial context exists. Ask them to predict the two outcomes before running the cell. Transition: run the full packet exercise, then inspect its assertions.

Executed tutorial: Classroom_Labs, L0–L12

Next: Hands-on: read back, reject, inspect

SLIDE 119 · O05

Hands-on: read back, reject, inspect

> **Use L12 to distinguish scope, identity and acknowledgment.**

- Run L12 after its earlier cells
- Find the full-scope HOLD
- Inspect the wrong-ID rejection
- Confirm no equipment dispatch

Give learners four minutes and keep this slide visible. First run the unmodified L12 cell after the earlier classroom cells. Inspect the displayed packet for the full MV/LV requirement and compare it with the explicitly MV teaching packet. Then inspect wrong_id in a scratch cell; it is intentionally not the accepted readback. Finally check the three assertions at the end of L12. Have partners explain the difference between an eligible packet and an acknowledged packet. Avoid changing the original assertion expectations during the first run. For an extension, discuss which fields would change if the requested control changed and which evidence would need to be recomputed. This is a local advisory exercise; the notebook has no equipment-dispatch endpoint. Transition: compare the predicted outcomes with the executed checks.

> **Experiment / prediction**
>
> Which of the three outcomes is permission to dispatch equipment?

Expected observation: None. Even the acknowledged packet is explicitly advisory.

Executed tutorial: Classroom_Labs, L0–L12

Next: The checks distinguish three outcomes

The checks distinguish three outcomes

> **Scope mismatch holds; wrong identity rejects; valid readback remains advisory.**

Exact excerpt: Classroom_Labs.ipynb, L12, lines 13–15. Run the complete cell in context.

```
assert full_scope['state'] == 'HOLD' and not full_scope['acknowledged']
assert wrong_id['acknowledgment_result'] == 'REJECTED'
assert readback['acknowledged'] and not readback['equipment_dispatch']
```

- Full MV/LV requirement: HOLD
- Wrong identifier: REJECTED
- Matching MV readback: acknowledged
- equipment_dispatch: false

Show the assertions after learners have made their predictions. They are deliberately more precise than an all-purpose PASS label. The full-scope packet is held because the supporting model is narrower than the request. The wrong identifier is rejected because it does not name the current packet. The eligible MV packet can be acknowledged, yet its equipment-dispatch flag remains false. These outcomes demonstrate the logic of the teaching application, not authentication, field authorization or human comprehension. Ask a participant to explain why the first failure cannot be repaired by entering the right identifier. It requires suitable evidence for the requested scope or an explicitly changed request, followed by the corresponding review. Transition: examine what happens when the underlying request changes.

Executed tutorial: Classroom_Labs, L0–L12

Next: A changed request needs new evidence

SLIDE 121 · O07

A changed request needs new evidence

> **A hash links content; it does not establish physical truth.**

- Change a limit or candidate
- Old binding no longer matches
- Recompute the affected study
- Review the new packet

Connect this question to L1 and L7, where the classroom explicitly checks stale bindings after a request or candidate changes. The binding helps detect a mismatch between saved evidence and its stated inputs. It does not prove that a sensor is trustworthy, that a model is correct or that an operator is authorized. Ask learners to identify the smallest changed field that should invalidate the old check: a current limit, load scale, topology revision or control setting all provide concrete examples. Their answer should include the affected numerical step, not only a request for a new language explanation. The important habit is to preserve evidence provenance through revision. Transition: apply this habit to three operator decision cards.

> **Discussion**
>
> Can the old acknowledgment survive a changed control?

Expected observation: The changed candidate needs new checks and a new matching review record.

Executed tutorial: Classroom_Labs, L0–L12

Next: Capstone: approve review, hold or request a study?

Capstone: approve review, hold or request a study?

> **Name the missing evidence and the next responsible step.**

- A: MV PASS; full-feeder AC FAIL
- B: restoration PASS; isolation record absent
- C: no alarm; protected measurements absent

Give each group one card and ask for a decision plus a concrete next step. Card A requires repair or another study at the requested model scope; a confident Llama sentence cannot override the AC failure. Card B exposes a missing operational prerequisite: a feasible mathematical switching plan is not proof of authorized isolation. Card C exposes incomplete information: an unalarmed residual alone cannot establish trustworthy state estimation under the constructed stealth attack. In each response, separate the numerical tool that can help from the authority or external evidence that the tool cannot invent. Do not assume that retaining an old setting is always safe after a changed topology or measurement regime. Use the final minute to compare the three different gaps. Transition: summarize how all five stages contribute.

> **Discussion**
>
> For your card, identify one missing fact and the next tool or human step.

Expected observation: A: broader-model repair. B: authorized isolation evidence. C: justified independent measurements or escalation.

Executed tutorial: Classroom_Labs, L0–L12 · Liu, Ning & Reiter, false-data injection, CCS 2009

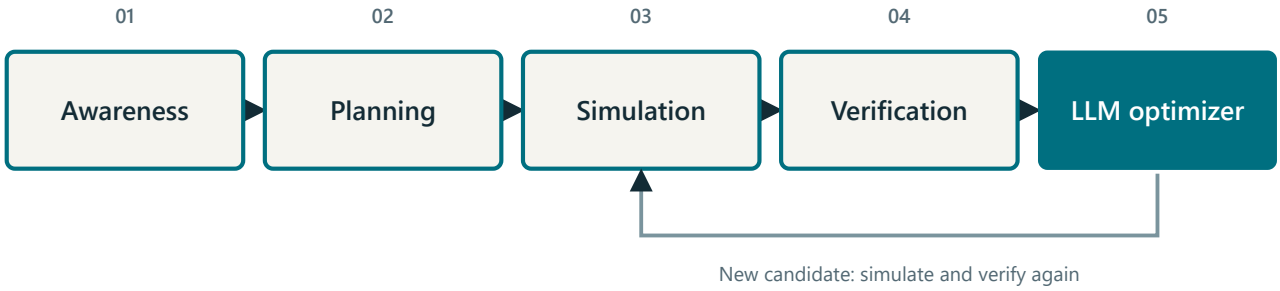
Next: Five stages, three cases, one evidence discipline

Synthesis

SLIDE 123 · S01

Five stages, three cases, one evidence discipline

> **Each stage contributes a method and a checkable output.**



Completed teaching diagram.

- Awareness: establish the study context
- Planning: generate admissible candidates
- Simulation + verification: test and accept
- Optimizer: refine; operator: review

Invite one learner per use case to trace a single candidate through the five proposed stages. Awareness establishes measurements, constraints and model scope. Planning calls the relevant numerical formulation. Simulation computes the consequence of candidate controls; verification checks the declared requirements and binds the result. The optimizer can request a bounded refinement or retain an incumbent, after which the relevant evidence is checked again. Human review is the subsequent authority boundary, not a sixth language agent. For voltage this includes MPC and SAC candidate comparison; restoration uses a weighted service formulation; cyber response distinguishes estimation evidence from protected-information assumptions. The methods change while the handoff discipline remains. Transition: choose how much orchestration the task actually requires.

Executed tutorial: Classroom_Labs, L0–L12 · Recorded voltage pilot: E1–E8

Next: Choose the simplest sufficient workflow

SLIDE 124 · S02

Choose the simplest sufficient workflow

> **Add an agent only when it addresses a measured task need.**

- Repeated calculation → fixed workflow
- Varied requests → typed LLM interface
- Independent studies → specialist coordination
- Consequential recommendation → enforced review

Treat this as a design decision rather than a contest between labels. A stable calculation with known inputs can be well served by a fixed workflow. Language interpretation becomes useful when users express varied requests, but the resulting tool calls still need validation. Specialist coordination can help when responsibilities and evidence streams differ; it also adds communication overhead and potential handoff failures. Recent agent-scaling research motivates testing that tradeoff instead of assuming that a larger team is better. The classroom direct-Python and LangGraph comparison is deliberately equivalent on its toy cases: orchestration structure alone should not change the mathematics. Ask participants which part of their own workflow is currently variable or error-prone. Transition: preserve enough information to make that improvement measurable.

Kim et al., scaling agent systems (2026) · Executed LangGraph teaching lab: G0–G5

Next: A reproducible result carries its context

SLIDE 125 · S03

A reproducible result carries its context

> **Archive the inputs, methods, outputs and evidence mode together.**

- Case, model scope and data revision
- Solver, policy and LLM versions
- Seeds, budgets and measured durations
- Fresh, archived or optional-live label

Use this checklist to prepare a reproducible scientific report, not a new permission process. The code cells already distinguish fresh numerical calculations, inference from saved SAC weights and archived actual LLM responses. The optional local Llama call is disabled by default, and its saved validation record includes a timestamp and timing fields. A screenshot of an answer is weaker than the structured record that identifies its request, model and tool results. Include failures and rejected alternatives where they affect interpretation. For the selector pilot, preserve the training, validation and held-out split and state that test-time learning was frozen. Ask learners to locate one such field in the supplied outputs. Transition: turn the remaining limitations into a clear next experiment.

Executed tutorial: Classroom_Labs, L0–L12 · Recorded voltage pilot: E1–E8 · Ollama chat API: timing fields

Next: Design your next experiment

SLIDE 126 · S04

Design your next experiment

> **Change one factor and state what would falsify your claim.**

- Hypothesis: name the expected benefit
- Control: retain a strong fixed baseline
- Measure: outcome, failures and latency
- Boundary: state the model and task scope

Ask every participant to write one sentence of the form: adding this mechanism improves this measurable outcome under these conditions. Good candidates include fewer stale-evidence reports, more correct tool selection for varied language requests, or a better tradeoff between objective and computation under a fixed trial budget. A weak hypothesis simply says that agentic AI is better. Require a baseline that can plausibly win and a condition under which the proposed benefit would not be supported. Suggest a held-out scenario split rather than retuning after viewing every result. If the claim concerns operator workload, the next experiment needs a human study rather than another simulation plot. Transition: use the exit ticket to show what you can explain and run now.

> **Experiment / prediction**
>
> Write one testable benefit and one result that would refute it.

Expected observation: Example: fewer unsupported reports at equal tool budget; equal or higher unsupported-report rate refutes the claimed benefit.

PowerAgentBench-SS (2026), §§II–III · Kim et al., scaling agent systems (2026)

Next: Exit ticket: explain, run, interpret

SLIDE 127 · S05

Exit ticket: explain, run, interpret

> **Connect one concept, one cell and one observed result.**

- Explain a DAG versus a bounded cycle
- Name a model-based and model-free method
- Interpret one failed candidate correctly
- Identify the operator acknowledgment boundary

Allow three minutes for an individual response followed by a brief exchange with a neighbor. Each learner should cite one notebook checkpoint and one actual observed result. For the graph question, a DAG has no directed cycle, while the implemented repair loop is a bounded cyclic state graph. MPC uses an explicit model during planning; the trained SAC policy supplies a continuous candidate from its learned mapping, and that action is still checked in a simulator. A failed candidate is neither proof that the entire framework failed nor proof of safety by itself; explain what the gate prevented and what remained unresolved. Finally identify why acknowledgment in the teaching dashboard does not dispatch equipment. Transition: return to the operator question that opened the tutorial.

> **Discussion**
>
> Which checkpoint would you show a colleague to explain the framework?

Expected observation: Choose a checkpoint and support the explanation with its exact inputs, output and stated limitation.

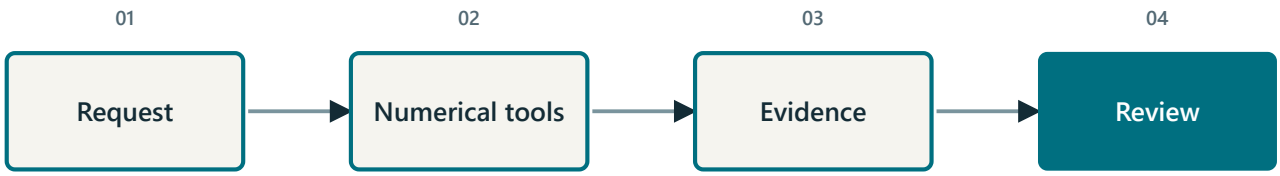
Executed LangGraph teaching lab: G0–G5 · Executed tutorial: Classroom_Labs, L0–L12

Next: From request to a reviewable recommendation

SLIDE 128 · S06

From request to a reviewable recommendation

> **Agentic assistance earns trust through useful tools, explicit evidence and accountable decisions.**



Completed teaching diagram.

- Language expresses the task
- Numerical methods produce candidates
- Checks determine what is supported
- People retain the review boundary

Close by asking the audience to describe the entire process in one sentence without using the word intelligent. A suitable description names the request, the numerical computation, the evidence checks and the operator review. The three cases have demonstrated different engineering formulations inside a shared orchestration pattern. They have also shown where a fixed method ties or beats a learned selector, where a reduced model misses full-feeder violations, and where a quiet residual is insufficient evidence. Those limits make the tutorial scientifically useful. Direct learners to the integrated handout for the symbol definitions, exact code references, exercise answers and citations, and to the Slidev source for the animated teaching sequence. The next step is a focused experiment on a clearly defined workflow need.

Executed tutorial: Classroom_Labs, L0–L12 · Executed LangGraph teaching lab: G0–G5 · Recorded voltage pilot: E1–E8

Next: Workshop complete